funnelling

Marco Servetto
School of Engineering and Computer Science, Victoria University of Wellington, New Zealand
Colin S. Gordon
School of Computer & Information Sciences, Drexel University, United States of America

Abstract

...

2012 ACM Subject Classification Replace ccsdesc macro with valid one

Keywords and phrases Dummy keyword

Funding *Colin S. Gordon*: US NSF CISE Award CCF-2007582

Acknowledgements I want to thank ...

1 Introduction

Java includes a useful feature called *method-local inner classes*, which has surprisingly subtle consequences. Like anonymous inner classes, this is a way for a method in a Java program to declare a class whose details are defined locally, entirely within the method, when the identity of the class is not needed outside the method's scope. However, these local classes are named, which allows them to be used with other language features exploiting nominal types. For example, one can cast objects to the method-local class type.

Unfortunately, as Servetto et al. [4] pointed out, there is an unsound interaction with generics. A non-generic method-local inner class `MyFoo` written in a generic method, or in a method of a generic class, can implicitly see the generic types in scope. Instances of `MyFoo` can then escape the lifetime of their enclosing method, where the generics had a certain runtime instantiation, and re-enter a second call of the same enclosing method, where the generics could have a different runtime instantiation. Servetto et al. probed the behaviour of JVM languages that support the same feature (Scala and Kotlin) and found that these had the same unsoundness problem for some use cases, but did flag concerns in other cases. However, this was not systematically investigated.

In this paper we clearly articulate the problem they observed, characterize the key properties that lead to the issue, and propose a static solution for future languages.

Example 1 demonstrates the issue.

```
// Example 1
public class Broken {
  static <X> Foo<X> m(Object other, final X x){
    class MyFoo extends Foo<X>{ public X x(){ return x; } }
    if(!(other instanceof MyFoo)){ return new MyFoo(); }
    return ((MyFoo)other); // cast passes with no warnings
  }
  public static void main(String[] a) {
    Foo<Integer> r1= Broken.<Integer>m("anything",5);
    Foo<String> r2= Broken.<String>m(r1,"Hi");
    System.out.println(r1.x()); //5
    System.out.println(r2.x()); //Throws ClassCastException: ..
  }
}
class Foo<X>{ public X x() {return x();} }
```

Line 4 declares a method-local inner class `MyFoo`, which depends on the generic parameter instantiation for `X` when the method is invoked. The `main` method invokes `m` twice. Line 9 invokes it for `Integer`s. Since the `other` argument for that call is not a `MyFoo`, the first invocation returns a `MyFoo` which is a `Foo<Integer>`. This result is then passed back into the second call to `m` on Line 10. This time, the `instanceof` check succeeds, because `other` in this case *is* a `MyFoo`, so the method (successfully) casts the `other` argument to `MyFoo`. Since `MyFoo` is declared as extending `Foo<X>`, this is a valid result for the function. Thus the `Foo<Integer>` that was passed as the first argument of the second call, on Line 10, is returned as a `Foo<String>`.

Clearly, this is wrong, because there are now two aliased variables of incompatible types; consequently, the later code fails. The failure itself is contrived. The two `println` calls actually dispatch to different overloads: the first to a version accepting an `Object`, and the second to the more specific overload expecting a `String`. The second call passes a value from a generic API to the specific overload, and this causes the Java compiler to insert the failing `checkcast` in the generated bytecode.

explain why (raw types)

A different version of Java could stop this problem at four points, two at compile time and two at run time:

- The `instanceof` check could have been statically flagged.
- The cast could have been statically flagged.
- The `instanceof` could have returned false in the second invocation, since it was used on a `MyFoo` from a different generic parameter instantiation.
- The cast could have failed, again because it was used on a `MyFoo` from a different generic parameter instantiation.

The source of this unsoundness is that the Java compiler does not treat `MyFoo` as generic. The generic type is seen as 'in scope', and it is 'captured' exactly as the values of local variables are captured. Within a single execution of the enclosing method, this is safe, because the type is constant. A `MyFoo` instance is conceptually an instance of a class that captures the generic type that is in scope when the instance is created. But when such a `MyFoo` instance escapes one execution of the enclosing method and enters a second execution of the same method, the instance is still logically connected to the same class type, with the original captured generic type. However, both the static type system and the run-time checks of casts and `instanceof` treat it as if it were connected to the new class type, with the new captured generic type.

Generalizing closure-capture behaviour to generic types as described above feels natural, but it is unsound in many situations. Closure capture is intrinsically private: a closure does not show in its external type signature what it has captured. Here it ends up being so private that the type system and the runtime have no access to it either. The static type system has to work on the publicly visible type, and in Java that type is not generic. This makes the compiler assume that the simplest nominal type identity is sufficient for casts and `instanceof` checks.

This problem touches on Java's erasure of generics, but only indirectly: even languages with reified generics store reified generics for the generic arguments exposed to the type system: a type like `List<Foo>` would get one reified generic, `Foo`, together with some language-mediated API to access it. A type like `MyFoo` would get no reified generics, since the generic type would have been captured and not exposed by the type. Thus, we argue that erasure versus reification is only marginally connected to this problem. The conceptual hurdle is in

the distinction between propagating the scope of generic types explicitly or implicitly.[1]

It is tempting to accept this as a quirk of Java, which is already known to have other sources of unsoundness [1]. However, this problem is broader in both scope and impact, as we show in the rest of the paper. This issue affects the statically typed JVM languages that support local classes, including Scala and Kotlin. It also propagates to compilers of those languages that do not target the JVM, such as Scala Native , and to ahead-of-time compilation of JVM bytecode, such as GraalVM's Native Image.

cite Scala Native

In the remainder of this paper:

- We detail the breadth of the problem, and how it is handled by various existing language implementations.
- We develop a core language modelling Java's generics, method-local classes and type tests, and show that typing method-local classes as Java does is unsound.
- We develop a new type system mechanism for systematically tracking dependence of a method-local class on generic parameters, and show that it soundly prevents the problem. Our solution, *funnelling*, makes the dependency of a method-local class on the generic types in scope explicit in its type: in Example 1 above, `MyFoo` would funnel `X` and become `MyFoo<X>`.

2 More Failing Examples

Example 1 is one of a family of programs that Java accepts with no warning and that fail at run time. This section shows more programs of this family.

2.1 Example 2: No Generic Supertype

In Example 1, the local class `MyFoo` mentions `X` in its supertype `Foo<X>`. Example 2 exercises the same fault without needing a generic supertype. Here `MyFoo` extends `Object`, and `X` occurs only in the return type of the method `x()` of `MyFoo`. The method `m` returns `Object`, so no type outside `m` mentions `X`.

```
1  // Example 2
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      if(!(other instanceof MyFoo)){ return new MyFoo(); }
6      MyFoo local= (MyFoo)other; // cast passes with no warnings
7      System.out.println("old was "+f.apply(local.x())); //ClassCastException
8      return local;
9    }
10   public static void main(String[] a) {
11     Object r1= Broken.<Integer>m("anything",5,i->""+i);
12     Broken.<String>m(r1,"Hi",s->s);
13   }
14 }
```

[1] Since JDK 16, with `Object o`, `o instanceof Foo<Integer>` is a compile-time error (before JDK 16, every non-reifiable `instanceof` target was an error), while the cast `(Foo<Integer>)o` is an unchecked warning; `instanceof` never warns. Modern Java accepts `o instanceof C<Ts>` only when the static type of `o` determines `Ts`; in this way an erased and a reified runtime give the same answer, as long as the program has no heap pollution. Example 1 creates heap pollution without any warning: `r2` has static type `Foo<String>`, but it refers to a `Foo<Integer>`.

The first call of `m` instantiates `X` with `Integer`, and returns a `MyFoo` whose method `x()` returns `5`. The second call of `m` instantiates `X` with `String`. In the second call, `other instanceof MyFoo` is true, and the cast `(MyFoo)other` at Line 6 succeeds with no warning. Then `local.x()` has static type `String`, but returns the `Integer 5`, and `f.apply(local.x())` at Line 7 throws a `ClassCastException`.

consider moving this level of detail (the cast is in a generated synthetic method) to the appendix. Maybe just say "the cast is in generated code" and put the rest in an appendix? Or footnote?

No instruction of `m` performs the failing cast. `javac` compiles the body of the lambda `s->s` into a private static method `lambda$main$1(String)` of the class `Broken`. At run time, the first evaluation of the lambda `s->s` calls `LambdaMetafactory`, which generates a class implementing `Function`; we call this class the *lambda class*. The method `apply(Object)` of the lambda class casts its argument to `String`, and then calls `lambda$main$1` with the result of that cast. Thus `f.apply(local.x())` passes the `Integer 5` to the method `apply` of the lambda class, and the cast to `String` in that method throws the `ClassCastException`.

The stack trace of the `ClassCastException` hides this. `LambdaMetafactory` defines the lambda class as a hidden class, and the JVM omits the frames of the methods of hidden classes from stack traces, unless the JVM runs with the diagnostic option `-XX:+ShowHiddenFrames`. So the topmost frame of the stack trace is the frame of `m`, at Line 7. The bytecode of `m` at Line 7 does contain a `checkcast` instruction to `String`, on the result of `f.apply`, but that instruction is never reached. The failure is detected in the lambda class: in code that the programmer did not write, at a point that depends on how javac and the JVM implement lambdas.

2.2 Example 3: No Explicit Cast

Example 3 exercises the same fault without needing an explicit cast. Java accepts Example 3 with no warning. Here we rely on the `other instanceof MyFoo local` test to bind `local` in one step.

```
1  // Example 3
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      if(other instanceof MyFoo local){ // no cast, no warnings
6        System.out.println("old was "+f.apply(local.x())); //ClassCastException
7      }
8      return new MyFoo();
9    }
10   public static void main(String[] a) {
11     Object r1= Broken.<Integer>m("anything",5,i->""+i);
12     Broken.<String>m(r1,"Hi",s->s);
13   }
14 }
```

As in Example 2, the method `apply` of the lambda class throws the `ClassCastException`.

2.3 Example 4: Pattern Matching in `switch`

Colins ays: this should be one sentence after previous example that a switch can trigger the same thing, and ref to an appendix w/ code

Example 4 exercises the same fault with a `switch` instead of an `instanceof`. Here the case `case MyFoo local` performs the same type test as the pattern `other instanceof MyFoo local` of Example 3.

```
1  // Example 4
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{ public X x(){ return x; } }
5      switch(other){
6        case MyFoo local: // no cast, no warnings
7          System.out.println("old was "+f.apply(local.x())); //ClassCastException
8          break;
9        default:
10         break;
11     }
12     return new MyFoo();
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->""+i);
16     Broken.<String>m(r1,"Hi",s->s);
17   }
18 }
```

2.4 Example 5: No `checkcast` on the Result

this is redundant with example 2's lambda discussion. Maybe use this code for three examples: Object vs. generic type, switch instead of explicit instanceof, and lambda moving cast failure to synthesized method. actually the actual lambdas are different. Can we fold example 5 into example 2 by changing line 12 to ""+s.length"?

Example 5 exercises the same fault without a `checkcast` instruction on the result of `f.apply` in the bytecode of `m`. Example 5 could be expressed with either the `switch` of Example 4 or the `instanceof` of Example 3; here we chose the switch, but the other way would make the same point. Example 5 uses `Function<X,Integer>` and the lambda `s->s.length()` instead of `Function<X,String>` and the lambda `s->s`.

```
1  // Example 5
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,Integer> f){
4      class MyFoo{ public X x(){ return x; } }
5      switch(other){
6        case MyFoo local: // no cast, no warnings
7          System.out.println("old was "+f.apply(local.x())); //ClassCastException
8          break;
9        default:
10         break;
11     }
12     return new MyFoo();
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->i);
16     Broken.<String>m(r1,"Hi",s->s.length());
17   }
18 }
```

The failure is the failure of Example 2: the method `apply` of the lambda class casts its argument, the `Integer 5`, to `String` before calling `lambda$main$1(String)`. The difference is only in the bytecode of `m`. Here the result of `f.apply` has static type `Integer`, and javac passes that result to the string concatenation without casting it to `Integer`, since any object can be converted to a `String`. So the bytecode of `m` contains no `checkcast` instruction on the result of `f.apply`. In Examples 2 to 4, the bytecode of `m` contains a `checkcast` instruction to `String` on the result of `f.apply`, but that instruction is never reached.

javac inserts a `checkcast` instruction where a value of an erased generic type is used at a more specific type. In Example 1, such a `checkcast` instruction detects the failure: javac inserts it in `main`, before the call of the overload `println(String)`. So one could expect a program with no `checkcast` instruction on the result of `f.apply` to fail later, in some virtual call, or not to fail at all. Example 5 shows that this expectation is wrong. In Examples 2 to 5, the value with the wrong type is the argument of `f.apply`: `local.x()` has static type `String`, but returns the `Integer 5`. The method `apply` of the lambda class casts that argument to `String` before `m` receives any result from `f.apply`. Thus the failure is detected at the same point in Examples 2 to 5, whether or not the bytecode of `m` contains a `checkcast` instruction on the result of `f.apply`.

2.5 Example 6: `X` Only in Private Members

Example 6 exercises the same fault without `X` in the public signature of `MyFoo`. Here `MyFoo` extends `Object`, and its only public method `show` takes a `MyFoo` and returns a `String`. `X` occurs only in the type of the private field `v`, and in the body of `show`, through the captured `f`.

focus is not really about private members, but that MyFoo's public signature doesn't mention X. Could have line 6 just capture x rather than storing in a field. Could turn example 2 into a family where we replace one line of example 2 with this show() definition we get this new example where X doesn't appear in the public signature.

```
1  // Example 6
2  public class Broken {
3    static <X> Object m(Object other, final X x, Function<X,String> f){
4      class MyFoo{
5        private final X v= x;
6        public String show(MyFoo o){ return f.apply(o.v); }
7      }
8      MyFoo mine= new MyFoo();
9      if(other instanceof MyFoo local){ // no cast, no warnings
10       System.out.println("old was "+mine.show(local)); //ClassCastException
11     }
12     return mine;
13   }
14   public static void main(String[] a) {
15     Object r1= Broken.<Integer>m("anything",5,i->""+i);
16     Broken.<String>m(r1,"Hi",s->s);
17   }
18 }
```

The first call of `m` returns a `MyFoo` whose field `v` holds the `Integer 5`. In the second call of `m`, `mine` is a new `MyFoo` of the second call, and `local` is the `MyFoo` of the first call. At Line 10, `mine.show(local)` applies the `f` of the second call, the lambda `s->s`, to the field `v` of `local`, the `Integer 5`. As in Example 2, the method `apply` of the lambda class throws the `ClassCastException`. No type that a client of `MyFoo` can see mentions `X`, and still the two

instances of `MyFoo` disagree on `X`.

2.6 Counterexample 7: Class Generics

Counterexample 7 moves Example 3 into a generic class: `X` is a generic parameter of the class `Box` instead of a generic parameter of the method `m`.

```
1  // Example 7
2  class Box<X> {
3    final X x;
4    Box(X x){ this.x= x; }
5    Object m(Object other, Function<X,String> f){
6      class MyFoo{ public X x(){ return x; } }
7      if(other instanceof MyFoo local){ // error: Object cannot be safely cast to MyFoo
8        System.out.println("old was "+f.apply(local.x()));
9      }
10     return new MyFoo();
11   }
12 }
13 public class Broken {
14   public static void main(String[] a){
15     Object r1= new Box<Integer>(5).m("anything",i->""+i);
16     new Box<String>("Hi").m(r1,s->s);
17   }
18 }
```

Example 7 marks the boundary of the family: with class generics, javac does flag the fault. javac rejects Example 7 at Line 7, with the error `Object cannot be safely cast to MyFoo`. The method `m` is an instance method of `Box<X>`, so `MyFoo` is an inner class of `Box<X>`, with an enclosing instance of type `Box<X>`. javac types `MyFoo` as `Box<X>.MyFoo`, as javac types a member class `MyFoo` of `Box`. So `MyFoo` is generic in `X`, and `other instanceof MyFoo` is the uncheckable type test `other instanceof Box<X>.MyFoo`.

The cast is flagged too. The following method `m` replaces the method `m` of Example 7. The type test against `String` replaces the type test against `MyFoo`, which javac rejects, and the cast of Example 2 follows it.

```
1  // Example 7 with a cast: the method m of Box
2    Object m(Object other, Function<X,String> f){
3      class MyFoo{ public X x(){ return x; } }
4      if(other instanceof String){ return new MyFoo(); }
5      MyFoo local= (MyFoo)other; // warning: [unchecked] unchecked cast
6      System.out.println("old was "+f.apply(local.x())); //ClassCastException
7      return local;
8    }
```

javac accepts this program, but reports an unchecked warning for the cast at Line 5. At run time the cast succeeds, and the program fails as Example 2 fails. So with class generics the program fails only after an unchecked warning, as a program with the cast `(List<String>)o` may fail. In Examples 1 to 6 instead, javac reports nothing.

Thus for the generic parameters of the enclosing class, Java already exposes the captured generic types in the type of the local class, through the type of its enclosing instance. Java does not do the same for the generic parameters of the enclosing method: no type of the local class carries the instantiation of the generic parameters of a method.

3 Funnelling

A natural suspicion is that this problem is "simply" a consequence of generic erasure: a language that cannot distinguish the generic instantiations of a class at run time must have difficulties of this sort. There is some truth to this suspicion: javac can accept a cast that the JVM cannot check, because of erasure. However, javac does flag uncheckable type tests: for `o` of type `Object`, javac reports an unchecked warning for the cast `(List<String>)o`, and an error for `o instanceof List<String>`. javac reports nothing for Examples 1 to 6 because `MyFoo` is not generic: the type test of `other` against `MyFoo` has no generic argument to leave unchecked. So reification does not change the situation by itself.

this paragraph may duplicate counter-example 7 text, otherwise we should move it there

The implementations of method-local classes, and of the similar features of other languages, differ in the granularity of the identity of the local class. The identity of the local class matters in two places: in the type system, where it decides whether two types of the local class are the same type, and at run time, where it decides the result of casts and type tests. There are three granularities:

1. **Identity as Declaration Point**: The declaration of a local class is a single point in the source code. So the local class `MyFoo` is one class, and the identity of `MyFoo` is its declaration point, the same for every instantiation of `m`.
2. **Identity as Generic Instantiation**: Generic classes and generic methods are functions from generic arguments to concrete classes and methods: `static <X> Object m(..)` declares infinitely many methods `m<Integer>`, `m<String>`, and so on, one for each instantiation of `X`. In the same way, the generic class declaration `class Foo<X>` declares infinitely many classes `Foo<Integer>`, `Foo<String>`, and so on. So each method `m<T>` declares its own local class `MyFoo`, since the body of each `m<T>` is the body of a different method.
3. **Identity as Dynamic Evaluation**: A declaration creates a new class, even for the same instantiation of the generic parameters. Scala has this granularity for captured values instead of captured types. An instance of an inner class `Inner` captures its enclosing instance `$outer`, and its type is the path-dependent type `a.Inner`. For the pattern `case i: a.Inner`, the Scala compiler emits an "outer test" that compares the captured `$outer` with `a`.[2] This is sound, but it is stricter than needed to avoid the problem at hand.

cite Scala outer tests

For method generics, the Java type system assumes granularity (1), as the casts do, so javac reports nothing and the program fails at run time. If the type of a local class contained the enclosing method and its generic instantiation, as in `Box<T1>.m<T2>.MyFoo`, then javac would flag the problem for generic methods too, as it does for generic classes. A version of Java with reified generics could solve the mismatch in Example 7, between the granularity (1) of the casts and the granularity (2) of the types. But reification would not change Examples 1 to 6, unless the types of local classes take the form `A<T1>.m<T2>.MyFoo`.

Granularities (1), (2) and (3) are points in a space of design options, but other points lie between them. Funnelling, the approach we present in this paper, lies between (1) and (2): in Example 1, `MyFoo` funnels `X` and becomes `MyFoo<X>`. For `other` of type `Object`, the existing rules of Java then flag `other instanceof MyFoo<X>` with an error and `(MyFoo<X>)other` with an unchecked warning, because `Object` does not determine `X`. These are the two compile-time points of the introduction. With reification, the same type test would be accepted by the compiler, and it would return false in the second call of `m`.

[2] Even here the type system exposes what is captured: inside the class of `a`, `Inner` means `this.Inner`.

We now show funnelling on the core language of Section 4, a version of Featherweight Java [3] rich enough to express the problem. The core language has generics, type tests and local classes. A local class is written as an expression, like an anonymous class in Java, but with a name: `new MyFoo<Fs> extends Foo<Ts>(es){Ms}` declares the local class `MyFoo` and creates an instance of it. This brings method-local classes into an expression-based calculus, without modelling statements. The funnels `Fs` are the generic parameters of `MyFoo`: a funnel `X funnel` funnels the type variable `X` in scope. During reduction, `X funnel` becomes the ground type that instantiates `X`. The type test `if (e instanceof T x){ e1 } else { e2 }` binds `x` in `e1`; the core language has no casts. In the listings, we omit the empty `<>` of methods and calls, and we write `<X>` for `<X extends Object<>>`. This is Example 1 in the core language, without funnels:

```
class A<> extends Object<>{}
class B<> extends Object<>{ B<> onlyB(){ return new B<>(); } }
class Foo<X> extends Object<>{ X x(){ return this.x(); } }
class Broken<> extends Object<>{
  <X> Foo<X> m(Object<> other, X x){ return
    if (other instanceof MyFoo<> w){ w }
    else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
}
```

As in Java, the type of the local class is `MyFoo<>`, with no generic parameters. Section 4 presents the type rules and the small step reduction of the core language. The type rules of Section 4 encode funnelling. With those type rules, the example above is ill typed: the supertype `Foo<X>` of `MyFoo` mentions `X`, but `MyFoo` does not funnel `X`. Rule NEW-LOCAL-JAVA enables the unsound Java typing:

$$\text{NEW-LOCAL-JAVA}\quad \frac{\Delta \vdash C'\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts\texttt{>}(es) : _ \qquad \Delta;\Gamma \setminus \texttt{this}, C\texttt{<>}\ \texttt{this} \vdash Ms\ \text{Ok} \qquad \mathsf{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\} : C\texttt{<>}}$$

Rule NEW-LOCAL-JAVA uses the syntax `new C<> extends C'<Ts>(es){Ms}`. This syntax is an intermediate form between method-local classes and anonymous classes of Java. A method-local class `class C<> extends C'<Ts>{ C(){ super(es); } Ms }` is declared first, and then instantiated with `new C<>()`. An anonymous class `new C'<Ts>(es){Ms}` is declared and instantiated by a single expression. The syntax `new C<> extends C'<Ts>(es){Ms}` forms a (named) local class. A local class creates an instance as an anonymous class does, thus a local class is easy to use in an expression-based language. A local class also declares its own name, thus `instanceof` can test for a local class.

Rule NEW-LOCAL-JAVA types the methods Ms of the local class C under all the bounds Δ and all the variables Γ in scope, except `this`, which has type C`<>`. The example above is well typed using the type rules of Section 4 and rule NEW-LOCAL-JAVA. The main expression is the first line of the reduction below, and it has type `B<>`. We now show the small step reduction using the abbreviations `eA`, `vA` and `vB`, and omitting the renaming of bound variables:

```
eA = new Broken<>().<A<>>m(new Foo<A<>>(), new A<>())
vA = new MyFoo<> extends Foo<A<>>(){ A<> x(){ return new A<>(); } }
vB = new MyFoo<> extends Foo<B<>>(){ B<> x(){ return new B<>(); } }

   new Broken<>().<B<>>m(eA, new B<>()).x().onlyB()
→* new Broken<>().<B<>>m(vA, new B<>()).x().onlyB()
→* (if (vA instanceof MyFoo<> w){ w } else { vB }).x().onlyB()
→ vA.x().onlyB()
→ new A<>().onlyB()
```

The first call of `m` reduces `eA` to `vA` with rules SIMPLEADDSUPER, SIMPLEFOUND and IFELSE. The type of `vA` is `MyFoo<>`: the instantiation `A<>` of `X` shows only in the supertype `Foo<A<>>` and in the method `x`. In the second call of `m`, rule IFTHEN checks only that the class of `vA` is a subclass of `MyFoo`. Thus the second call returns `vA`, a `Foo<A<>>`, as a `Foo<B<>>`. The last expression calls `onlyB` on an `A<>`, and `A<>` does not have a method called `onlyB`. Rule SEARCHSUPER looks for `onlyB` up to `Object<>`, where no rule applies: the reduction is stuck.

With funnelling, `MyFoo` must funnel `X`, written `new MyFoo<X funnel>`, and instances of the local class are of type `MyFoo<X>`. Rule CAST-DOWN accepts a type test only when the static type determines `X`, as Java does. Thus `other instanceof MyFoo<X> w` is ill typed for `Object<> other`, and the method `m` is ill typed. This is the rule by which Java rejects `o instanceof List<String>` for `Object o`. If we change the signature of `m` as follows, the method `m` is well typed:

```
  <X> Foo<X> m(Foo<X> other, X x){ return
    if (other instanceof MyFoo<X> w){ w }
    else { new MyFoo<X funnel> extends Foo<X>(){ X x(){ return x; } } }; }
```

Now `eA` is a well typed expression of type `Foo<A<>>`, but the main expression is ill typed: the second call of `m` expects a `Foo<B<>>` as the first argument, but `eA` is a `Foo<A<>>`.

With reification, the funnelled method `m` with `Object<> other` would be well typed, but the reduction would change. In the second call of `m`, the type test of the `MyFoo<A<>>` created by the first call against `MyFoo<B<>>` is false. The second call returns a new `MyFoo<B<>>`, and the main expression reduces to `new B<>()`. Reification without funnelling does not help: if we allow the value `vA` to have type `MyFoo<>`, even a reified type test of `vA` against `MyFoo<>` is true.

4 A Core Language for Funnelling

We now present the core language of Section 3: its syntax, its small step reduction and its type rules. The type rules encode funnelling. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA gives the Java typing of local classes.

4.1 Syntax

Expressions	e	::=	$x \mid e.\texttt{<}Ts\texttt{>}m(es) \mid e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es) \mid e.f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$
			$\mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(es)\texttt{\{}Ms\texttt{\}}$
			$\mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}}$
Types	T	::=	$C\texttt{<}Ts\texttt{>} \mid X$
Funnels	F	::=	$G \mid X\ \texttt{funnel}$
Ground Types	G	::=	$C\texttt{<}Gs\texttt{>}$
Type Env.	Γ	::=	$T_1\ x_1, \ldots, T_n\ x_n$
Bounds	Δ	::=	$X_1\ \texttt{extends}\ T_1, \ldots, X_n\ \texttt{extends}\ T_n$
Declaration	Dec	::=	$\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ Ss\ \texttt{\}}$
(Field) Slots	S	::=	$T\ f\texttt{;}$
Methods	M	::=	$\texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}}$
Class Decl.	K	::=	$\text{Dec} \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\texttt{\{}Ms\texttt{\}}$

An expression is a variable x, a method call $e.\texttt{<}Ts\texttt{>}m(es)$, a super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$, a field access $e.f$, an object creation $\texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$ for a top-level type, a local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\texttt{\{}Ms\texttt{\}}$ or a type test $\texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}}$.[3] As in FJ, `this` is just a special case of a variable x. A method call $e.\texttt{<}Ts\texttt{>}m(es)$ calls the method m on the receiver e, with type arguments Ts and arguments es. A super call $e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es)$ searches the method m starting from the class of the type T, instead of from the class of the receiver e. That is, a super call works as a flexible kind of Java `super` call, attempting to call the method m as if the receiver were of the supertype T. An object creation $\texttt{new}\ C\texttt{<}Ts\texttt{>}(es)$ creates an instance of the top-level class $C\texttt{<}Ts\texttt{>}$. The arguments es are the values of the fields of C, followed by the arguments of the object creation of the superclass of C. A local class $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\texttt{\{}Ms\texttt{\}}$ declares the local class C and creates an instance of C.

The funnels Fs serve a double role. From the perspective of the inside of the local class (the methods Ms), the funnels Fs declare the generics visible in the local class. From the perspective of the `new` expression itself, the funnels Fs instantiate those generic parameters, forming the actual type of the created instance.

To allow a local class instance to exist in the main expression, a funnel is either a ground type G or a funnelled type variable $X\ \texttt{funnel}$. In Decls, all the funnels are of the form $X\ \texttt{funnel}$, by well-formedness requirement A11. In the main expression, outside of the methods of local classes, we expect all the funnels to be of the form G. When a local class is nested in a method of a local class in the main expression, a mixture of funnels $X\ \texttt{funnel}$ and G can be present.

A local class has no fields: the arguments es are the arguments of the object creation of the superclass $C'\texttt{<}Ts\texttt{>}$. The methods Ms of a local class can use the variables and the type variables in scope. The type test $\texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}}$ reduces to e_1, with x bound to the value of e_0, when the value of e_0 is an instance of T; otherwise the type test reduces to e_2.

A type is a class type $C\texttt{<}Ts\texttt{>}$ or a type variable X. A ground type G is a type with no type variables. A ground type is the equivalent of a value, but at the type level. The conventional type environment Γ gives a type to each variable in scope. Bounds Δ give an upper bound to each type variable in scope. A declaration $\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ Ms\ Ss\ \texttt{\}}$ declares

[3] The syntax `if (a instanceof B c)` is valid Java since Java 16.

the top-level class C with the type variables bounded by Δ, the superclass C'<Ts>, the methods Ms and the fields/slots Ss. A method <Δ>T_0 m(T_1 x_1, …, T_n x_n) { return e; } has the type variables bounded by Δ, the return type T_0, the parameters $x_1, \ldots, x_n$ and the body e. Every method has a body; for simplicity, as in FJ, there are no abstract methods. A class declaration K is either a top-level declaration Dec or a local class. The superclass of every class, top-level or local, is a top-level class: local classes cannot be extended.

A program is a class table Decls and a main expression e. We require the following well-formedness conditions:

(A1) In Decls, class names are unique across top-level and local classes.
(A2) In a class, all the field names, including the field names of the superclasses, are disjoint.
(A3) In a class, top-level or local, all the method names are disjoint.
(A4) In a method, all the parameter names $x_1, \ldots, x_n$ are disjoint.
(A5) In a Δ, all the X in dom(Δ) are disjoint.
(A6) In a list of funnels Fs, all the X in dom(Fs) are disjoint.
(A7) There is no shadowing of X or x, except for `this`.
(A8) No parameter and no variable of an `if` is `this`.
(A9) Decls contains `class Object<> extends Object<> { }`, and the chain of superclasses of every other top-level class reaches `Object`.
(A10) In Decls, the superclass of every class, top-level or local, is a top-level class.
(A11) In Decls, every funnel is of the form X `funnel`.

Well-formedness requirements A1-A10 are common requirements, such as uniqueness of names. To simplify scoping, no shadowing is allowed. A method body of a local class also binds `this`, thus shadowing the `this` of the enclosing method; this is the only form of shadowing.

4.2 Values and Contexts

$$
\begin{array}{rrcl}
\text{Values} & v & ::= & \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{)} \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}} \\
\text{Eval. Ctx.} & \mathcal{E} & ::= & \square \mid \mathcal{E}\texttt{.<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.<}Gs\texttt{>}m\texttt{(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
 & & & \mid \mathcal{E}\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}es\texttt{)} \mid v\texttt{.[}G\texttt{]<}Gs\texttt{>}m\texttt{(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
 & & & \mid \mathcal{E}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Gs\texttt{>(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)} \\
 & & & \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{,}\mathcal{E}\texttt{,}es\texttt{)\{}Ms\texttt{\}} \\
 & & & \mid \texttt{if (}\mathcal{E}\ \texttt{instanceof}\ G\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
\text{Expr. Ctx.} & \mathcal{C} & ::= & \square \mid \mathcal{C}\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.<}Ts\texttt{>}m\texttt{(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
 & & & \mid \mathcal{C}\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{)} \mid e\texttt{.[}T\texttt{]<}Ts\texttt{>}m\texttt{(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
 & & & \mid \mathcal{C}\texttt{.}f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)} \\
 & & & \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>(}es\texttt{,}\mathcal{C}\texttt{,}es'\texttt{)\{}Ms\texttt{\}} \\
 & & & \mid \texttt{if (}\mathcal{C}\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
 & & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ \mathcal{C}\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} \\
 & & & \mid \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ \mathcal{C}\ \texttt{\}}
\end{array}
$$

4.3 Declarations

Define K declared
Dec declared iff Dec ∈ Decls

`new` C_0<Fs> `extends` C'<Ts>(*es*){Ms} declared
where K declared
<_>_ m(_) { return e; } ∈ K
`new` C_0<Fs> `extends` C'<Ts>(*es*){Ms} occurs in e

Define $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts'$

$\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$
where $\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}\ \texttt{\{}\ _\ _\ \texttt{\}}$ declared

$\mathsf{scope}(C_0\texttt{<}Ts\texttt{>}) = Fs \overset{\Delta,\Delta'}{\Longleftrightarrow} Ts$
where $\texttt{<}\Delta'\texttt{>}_\ m\texttt{(_)}\ \texttt{\{}\ \texttt{return}\ e\texttt{;}\ \texttt{\}} \in C$
$\texttt{new}\ C_0\texttt{<}Fs\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>(_)\{_\}}$ occurs in e
$\mathsf{scope}(\mathrm{generic}(C)) = \Delta; _$

Define $Fs \overset{\Delta}{\Longleftrightarrow} Ts = \Delta'; Ts' \qquad (\Delta; Ts) \cdot (\Delta'; Ts') = \Delta''; Ts''$

$\emptyset \overset{\Delta}{\Longleftrightarrow} \emptyset = \emptyset; \emptyset$

$X\ \texttt{funnel}, Fs \overset{\Delta}{\Longleftrightarrow} T, Ts = (X\ \texttt{extends}\ \Delta(X); T) \cdot (Fs \overset{\Delta}{\Longleftrightarrow} Ts)$

$G, Fs \overset{\Delta}{\Longleftrightarrow} G, Ts = Fs \overset{\Delta}{\Longleftrightarrow} Ts$

$(\Delta; Ts) \cdot (\Delta'; Ts') = \Delta, \Delta'; Ts, Ts'$

Define $M \in K \qquad M \in C \qquad \mathrm{generic}(C) = T \qquad |F| = T$

$M \in K$ iff $M \in Ms$
where $K \in \{\texttt{class}\ _\texttt{<}_\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}\ \texttt{\{}\ Ms\ _\ \texttt{\}}, \texttt{new}\ _\texttt{<}_\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>(_)\{}Ms\texttt{\}}\}$

$M \in C$ iff $M \in K$
where K declared
$K \in \{\texttt{class}\ C\texttt{<}_\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}\ \texttt{\{}\ _\ _\ \texttt{\}}, \texttt{new}\ C\texttt{<}_\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>(_)\{_\}}\}$

$\mathrm{generic}(C) = C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}$
where $\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}\ \texttt{\{}\ _\ _\ \texttt{\}}$ declared

$\mathrm{generic}(C) = C\texttt{<}\mathrm{dom}(Fs)\texttt{>}$
where $\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>(_)\{_\}}$ declared

$|X\ \texttt{funnel}| = X$

$|G| = G$

By well-formedness requirement A1, a class name C identifies at most one K with K declared. $M \in K$ holds when M is a method of the class declaration K, top-level or local. $M \in C$ holds when M is a method of the declared class C, top-level or local. For a top-level class C with bounds Δ, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta; Ts$. For a local class C with funnels Fs, $\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0$, where Δ_0 is the subset of the bounds in scope in the body of the method m declaring C, restricted to $\mathrm{dom}(Fs)$ and in the order of $\mathrm{dom}(Fs)$; note that m could be a method of either a top-level or local class. Ts_0 is the subsequence of Ts, of the same size as $\mathrm{dom}(Fs)$, holding the type argument of each funnelled X. $\mathsf{scope}(_) = \Delta; Ts$ is expected to always return a Δ and a Ts of the same length. This is enforced at the usage points, where we use the notation $T[\Delta = Ts]$, the simultaneous substitution, defined only when Δ and Ts have the same length.

For a type variable X, $\Delta(X)$ is the bound of X in Δ. $\mathrm{dom}(\Delta)$ is the sequence of the type variables in Δ, and $\mathrm{cod}(\Delta)$ is the sequence of the bounds in Δ. $\mathrm{dom}(Fs)$ is the sequence of the X such that $X\ \texttt{funnel}$ is in Fs. $\mathsf{FTV}(T)$ is the set of the type variables occurring in T. The figures leave the following conventions implicit. A sequence $e_1, \ldots, e_n$, or es, separates its elements with commas; a comma also concatenates two sequences, and either sequence may be empty. A function or a judgement applied to sequences stands for one application at every position, and requires the sequences to have the same length: $|F_1, \ldots, F_n|$ is $|F_1|, \ldots, |F_n|$; $\mathsf{FTV}(T_1, \ldots, T_n)$ is $\mathsf{FTV}(T_1) \cup \ldots \cup \mathsf{FTV}(T_n)$; $\Delta \vdash T_1, \ldots, T_n$ Ok is $\Delta \vdash T_1$ Ok $\quad \ldots \quad \Delta \vdash T_n$ Ok; $\Delta \vdash T_1, \ldots, T_n <: T'_1, \ldots, T'_n$ is $\Delta \vdash T_1 <: T'_1 \quad \ldots \quad \Delta \vdash T_n <: T'_n$; $\Delta; \Gamma \vdash e_1, \ldots, e_n : T_1, \ldots, T_n$ is $\Delta; \Gamma \vdash e_1 : T_1 \quad \ldots \quad \Delta; \Gamma \vdash e_n : T_n$. Finally, $m \notin Ms$ holds when no method in Ms has name m, and $\mathrm{dom}(Ms)$ is the set of the names of the methods in Ms. $\Gamma \setminus x$ is Γ without the variable x. For a set of type variables Xs, $\Gamma|_{Xs}$ keeps the variables $T\ x$ of Γ such that $\mathsf{FTV}(T) \subseteq Xs$.

The pairing $Fs \overset{\Delta}{\Longleftrightarrow} Ts$ goes through the funnels Fs and the arguments Ts together, one position at a time, where $\emptyset$ is the empty sequence. At the position of a funnel X `funnel`, it adds X `extends` $\Delta(X)$ to the bounds and the argument to the types. At the position of a ground funnel G, the argument must be G itself, and nothing is added. The operator $(\Delta; Ts) \cdot (\Delta'; Ts')$ concatenates two results of a pairing, component by component. An expression e' occurs in e when $e = \mathcal{C}[e']$, for an expression context $\mathcal{C}$. Expression contexts $\mathcal{C}$ do not enter the methods of a local class; thus $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ has at most one value.

4.4 Operational Semantics

We now present the reduction $e \to e'$ of the main expression. The reduction is small step, and uses the values v and the evaluation contexts $\mathcal{E}$.

Define $\mathsf{typeOf}(v) = G$

$\mathsf{typeOf}(v) = C\texttt{<}Gs\texttt{>}$
where $v \in \{\texttt{new}\ C\texttt{<}Gs\texttt{>(_)}, \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ \texttt{_<_>(_)\{_\}}\}$

$$\boxed{e \to e'}$$

simpleFieldFound

$$\frac{\texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_>}\ \texttt{\{}\ \texttt{_}\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \texttt{\}}\ \mathsf{declared} \qquad f_i \in f_1, \ldots, f_n}{\texttt{new}\ C\texttt{<}Gs\texttt{>(}v_1, \ldots, v_n, vs\texttt{)}\,.\,f_i \to v_i}$$

simpleFieldSuper

$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ \texttt{_}\ T_1\ f_1\texttt{;}\ \ldots T_n\ f_n\texttt{;}\ \texttt{\}}\ \mathsf{declared} \qquad f \notin f_1, \ldots, f_n \qquad C \neq \texttt{Object}}{\texttt{new}\ C\texttt{<}Gs\texttt{>(}v_1, \ldots, v_n, vs\texttt{)}\,.\,f \to \texttt{new}\ C'\texttt{<}Ts[\Delta = Gs]\texttt{>(}vs\texttt{)}\,.\,f}$$

innerFieldSuper

$$\frac{}{\texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)\{}Ms\texttt{\}}\,.\,f \to \texttt{new}\ C'\texttt{<}Gs'\texttt{>(}vs\texttt{)}\,.\,f}$$

ifThen

$$\frac{\emptyset \vdash \mathsf{typeOf}(v) <: C'\texttt{<_>} \qquad \mathsf{fresh}(e_1, e_1')}{\texttt{if (}v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x\texttt{)}\ \texttt{\{}\ e_1\ \texttt{\}}\ \texttt{else}\ \texttt{\{}\ e_2\ \texttt{\}} \to e_1'[x = v]}$$

ifElse

$$\frac{\emptyset \nvdash \mathsf{typeOf}(v) <: C'\texttt{<_>}}{\texttt{if (}v\ \texttt{instanceof}\ C'\texttt{<}Ts\texttt{>}\ x\texttt{)}\ \texttt{\{}\ e_1\ \texttt{\}}\ \texttt{else}\ \texttt{\{}\ e_2\ \texttt{\}} \to e_2}$$

$$\textsc{simpleAddSuper}\ \frac{v = \texttt{new}\ C\texttt{<}Gs\texttt{>(_)}}{v\texttt{.<}Gs'\texttt{>}m(vs) \to v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(vs)}$$

$$\textsc{simpleFound}\ \frac{\begin{array}{c}\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ \texttt{_<_>}\ \{\ Ms\ _\ \}\ \mathsf{declared} \\ M \in Ms \qquad \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \{\ \texttt{return}\ e_0\texttt{;}\ \}) \\ e = e_0[\Delta, \Delta' = Gs, Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

$$\textsc{innerFound}\ \frac{\begin{array}{c}v_0 = \texttt{new}\ \texttt{_<_>}\ \texttt{extends}\ \texttt{_<_>(_)\{}Ms\texttt{\}} \\ M \in Ms \qquad \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{)}\ \{\ \texttt{return}\ e_0\texttt{;}\ \}) \\ e = e_0[\Delta' = Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

$$\textsc{innerSuper}\ \frac{v_0 = \texttt{new}\ \texttt{_<_>}\ \texttt{extends}\ C\texttt{<}Gs\texttt{>(_)\{}Ms\texttt{\}} \qquad m \notin Ms}{v_0\texttt{.<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(v_1, \ldots, v_n)}$$

$$\textsc{searchSuper}\ \frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ _\ \}\ \mathsf{declared} \qquad m \notin Ms \qquad C \neq \texttt{Object}}{v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(vs) \to v\texttt{.[}C'\texttt{<}Ts[\Delta = Gs]\texttt{>]<}Gs'\texttt{>}m(vs)} \qquad \textsc{ctx}\ \frac{e \to e'}{\mathcal{E}[e] \to \mathcal{E}[e']}$$

Most of the rules are standard. Rules simpleFieldFound, simpleFieldSuper and innerFieldSuper read a field. Rule simpleFieldFound reads a field declared by the class of the object. Rule simpleFieldSuper reads a field not declared by the class C of the object from the object of the superclass of C, made of the remaining values. A local class declares no fields, so rule innerFieldSuper reads every field from the object of the superclass.

Rules ifThen and ifElse test the class of the value v against the class C' of the type test, ignoring the type arguments Ts of $C'\texttt{<}Ts\texttt{>}$. Rule ifThen reduces to e_1 with x replaced by v; rule ifElse reduces to e_2.

Rules simpleAddSuper, simpleFound, innerFound, innerSuper and searchSuper call a method. Rule simpleAddSuper turns a call on an object of a top-level class into a super call starting from the class of the object. Rule simpleFound runs the method m found in the class of a super call, replacing the type variables of the class and of the method, `this` and the parameters. Rule innerFound runs the method m found in a value of a local class; the funnelled type variables are already replaced in that value, so only the type variables of the method are replaced. Rule innerSuper turns a call on a value of a local class without the method m into a super call starting from its superclass. Rule searchSuper moves a super call to the superclass when the class does not declare m. Rule ctx reduces inside an evaluation context.

Rules ifThen, simpleFound and innerFound also use the relation fresh. The names bound in an expression e are the variable of every `if` occurring in e, and the names bound in every method of a local class occurring in e. The names bound in a method M are the type variables of its signature, its parameters, and the names bound in its body. $\mathsf{fresh}(e, e')$ holds when e' is e where every name bound in e is renamed into a fresh name, that is a name occurring nowhere in the program; distinct binders get distinct fresh names. $\mathsf{fresh}(M, M')$ holds when M' is M renamed in the same way. A rule using fresh can choose any fresh names. The variable `this` is never renamed. Renaming is needed because a value of a local

class carries the code of its methods. For example, consider a local class `L` whose method `m` has a parameter `x` and calls `m` on a captured value `y` of class `L`:

```
Object<> k(L<> y){ return
  new L<> extends Object<>(){ Object<> m(Object<> x){ return y.m(x); } }; }
```

In a call of `k` with a value `v` of class `L`, the body of `m` becomes `v.m(x)`, and `v` contains a copy of the method `m` binding `x` again. Without renaming, rule INNERFOUND would replace `x` also in the method `m` of `v`. Without fresh, the substitution would attempt to place a binder of a name under a binder of the same name, breaking well-formedness requirement A7. Rules IFTHEN, SIMPLEFOUND and INNERFOUND place values under the binders of the expression they substitute into. Those binders are fresh, so the values do not bind the same names. A substitution can copy a value, so several copies of a value can bind the same names; those copies are never nested, so they do not shadow each other. Thus a substitution never reaches a type variable or a variable bound inside the substituted expression.

4.5 Typing

$$\boxed{\Delta;\Gamma \vdash e : T}$$

SUBS
$$\frac{\Delta;\Gamma \vdash e : T_1 \qquad \Delta \vdash T_2 \text{ Ok} \qquad \Delta \vdash T_1 <: T_2}{\Delta;\Gamma \vdash e : T_2}$$

VAR
$$\frac{}{\Delta;\Gamma \vdash x : \Gamma(x)}$$

NEW-OBJECT
$$\frac{}{\Delta;\Gamma \vdash \texttt{new Object<>()} : \texttt{Object<>}}$$

CALL
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts_0\texttt{>} \\ \mathsf{scope}(C_0\texttt{<}Ts_0\texttt{>}) = \Delta_0; Ts_2 \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ _ \}} \in C_0 \\ \forall i \in 0..n.\, T_i' = T_i[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0\texttt{.<}Ts_1\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0'}$$

DECLFIELD
$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e : C\texttt{<}Ts'\texttt{>} \\ \texttt{class}\ C\texttt{<}\Delta'\texttt{>}\ \texttt{extends}\ \texttt{_<_> \{ _}\ Ss\ \texttt{\}}\ \text{declared} \\ T\ f\texttt{;} \in Ss\end{array}}{\Delta;\Gamma \vdash e.f : T[\Delta' = Ts']}$$

CALL-LB-RUNTIME
$$\frac{\begin{array}{c}\Delta \vdash C\texttt{<}Ts\texttt{>} <: C_0\texttt{<}Ts_0\texttt{>} \\ \texttt{class}\ C_0\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ \texttt{_<_> \{ _ _ \}}\ \text{declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ _ \}} \in C_0 \\ \forall i \in 0..n.\, T_i' = T_i[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \texttt{class}\ C\texttt{<_>}\ \texttt{extends}\ \texttt{_<_> \{ _ _ \}}\ \text{declared} \\ \Delta \vdash C\texttt{<}Ts\texttt{>} \text{ Ok} \qquad \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \Delta;\Gamma \vdash e_0, \ldots, e_n : C\texttt{<}Ts\texttt{>}, T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0\texttt{.[}C\texttt{<}Ts\texttt{>]<}Ts_1\texttt{>}m\texttt{(}e_1, \ldots, e_n\texttt{)} : T_0'}$$

NEW-TOP

$$\frac{\texttt{class}\ C\texttt{<}\Delta'\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ _\ T_1\ f_1\texttt{;}\ \ldots\ T_n\ f_n\texttt{;}\ \}\ \text{declared} \qquad \Delta \vdash C\texttt{<}Ts'\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash e_1,\ldots,e_n : T_1,\ldots,T_n[\Delta' = Ts'] \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts[\Delta' = Ts']\texttt{>(}es\texttt{)} : _}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Ts'\texttt{>(}e_1,\ldots,e_n,es\texttt{)} : C\texttt{<}Ts'\texttt{>}}$$

NEW-LOCAL

$$\frac{\begin{array}{c}\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}\ \text{consistent} \\ Fs \overset{\Delta}{\Longleftrightarrow} |Fs| = \Delta';_ \qquad \mathsf{FTV}(\mathrm{cod}(\Delta')) \subseteq \mathrm{dom}(\Delta') \\ \Delta' \vdash C\texttt{<}|Fs|\texttt{>}\ \text{Ok} \qquad \Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \\ \Delta';\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}, C\texttt{<}|Fs|\texttt{>}\ \texttt{this} \vdash Ms\ \text{Ok} \\ \mathsf{overrideOk}(\Delta', C_0\texttt{<}Ts\texttt{>}, Ms)\end{array}}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<}|Fs|\texttt{>}}$$

IF-INSTANCEOF

$$\frac{\Delta;\Gamma \vdash e_0 : T_0 \qquad \Delta \vdash T\ \text{Ok} \qquad \Delta \vdash T_0 \rightsquigarrow T \qquad \Delta;\Gamma, T\ x \vdash e_1 : T' \qquad \Delta;\Gamma \vdash e_2 : T'}{\Delta;\Gamma \vdash \texttt{if (}e_0\ \texttt{instanceof}\ T\ x\texttt{) \{}\ e_1\ \texttt{\}}\ \texttt{else \{}\ e_2\ \texttt{\}} : T'}$$

$$\boxed{\Delta \vdash T \rightsquigarrow T'}$$

CAST-UP

$$\frac{\Delta \vdash T <: T'}{\Delta \vdash T \rightsquigarrow T'}$$

CAST-VAR

$$\frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T \rightsquigarrow T'}{\Delta \vdash X \rightsquigarrow T'}$$

CAST-DOWN

$$\frac{\Delta \vdash C\texttt{<}Ts\texttt{>} <: C'\texttt{<}Ts_0\texttt{>} \qquad \mathrm{generic}(C) = C\texttt{<}Xs\texttt{>} \qquad \emptyset \vdash C\texttt{<}Xs\texttt{>} <: C'\texttt{<}Ts'\texttt{>} \qquad Xs \subseteq \mathsf{FTV}(Ts')}{\Delta \vdash C'\texttt{<}Ts_0\texttt{>} \rightsquigarrow C\texttt{<}Ts\texttt{>}}$$

Define K consistent

$\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}$ consistent

where $\texttt{new}\ C\texttt{<}Fs'\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts'\texttt{>(_)\{}Ms'\texttt{\}}$ declared

$Ts = Ts'[Fs' = |Fs|]$

$\mathrm{dom}(Ms') = \mathrm{dom}(Ms)$

$\mathsf{overrideOk}(C\texttt{<}|Fs|\texttt{>}, Ms)$

The rule NEW-LOCAL is the heart of funnelling, including subtle management of type environments. The rule constructs a restricted set of type bounds and variable bindings reflecting only the explicitly-funnelled type variables: Δ' includes only the explicitly-funnelled type variables, and $\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}$ keeps only the variables of Γ, other than `this`, whose types mention only type variables of the result type $C\texttt{<}|Fs|\texttt{>}$ (which must be well-kinded under Δ'). The bounds Δ' are computed by $Fs \overset{\Delta}{\Longleftrightarrow} |Fs|$, which is undefined when a funnelled type variable is not in Δ. However, the rule still uses the *full* ambient environments for checking certain subexpressions. This is because some $e \in es$ may initially contain uses of *non*-funnelled type variables (e.g., in the type arguments of a method call), but will reduce to a value which no longer depends on such type variables. This is why the premise $\Delta;\Gamma \vdash \texttt{new}\ C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _$ checks the effective supertype instance under $\Delta;\Gamma$, but the premise $\Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \text{Ok}$ ensures the resulting type of that expression satisfies the funnelling constraints by checking kinding under the more restrictive Δ'.

A local class occurs in two places: (1) The (original) declaration of the local class is written in the body of a method of a top-level class, possibly nested inside other local classes. (2) Copies of the local class can be in the main expression: reduction copies the declaration from the class table into the main expression, replacing type variables with types and variables with values.

The relation K consistent trivially holds for case (1). For case (2), the first condition of

the definition finds the declaration of K. The supertype and the method signatures of K must be those of the declaration, with the funnelled type variables of Fs' replaced by $|Fs|$. K and the declaration have the same method names.

Rules CALL and SUB-INHERIT-LOCAL read the declaration, while the reduction runs the methods of the copy.

The relation $\Delta \vdash T \rightsquigarrow T'$ is the *checked cast*: a type test against T' on a value of static type T is decided by the class of the value alone. Rule IF-INSTANCEOF accepts the type test `if (`e_0 `instanceof` T x`) {` e_1 `} else {` e_2 `}` only when $\Delta \vdash T_0 \rightsquigarrow T$, where T_0 is the type of e_0. Rule CAST-UP accepts a target T' that is a supertype of the source T. Rule CAST-VAR replaces a source type variable X with the bound of X. Rule CAST-DOWN accepts a target $C\texttt{<}Ts\texttt{>}$ that is a subtype of the source $C'\texttt{<}Ts_0\texttt{>}$, when the source determines the type arguments Ts. $\text{generic}(C) = C\texttt{<}Xs\texttt{>}$ is the class C instantiated with its own type variables Xs, and $C'\texttt{<}Ts'\texttt{>}$ is the supertype of $C\texttt{<}Xs\texttt{>}$ with class C'. The premise $\emptyset \vdash C\texttt{<}Xs\texttt{>} <: C'\texttt{<}Ts'\texttt{>}$ of rule CAST-DOWN is correctly called with $\emptyset$ as the generic bounds: a subtyping derivation between class types uses only rules SUB-REFL, SUB-INHERIT and SUB-INHERIT-LOCAL, which do not read Δ. Every type variable of Xs occurs in Ts', thus the type arguments Ts_0 of the source fix the type arguments Ts of the target. Thus a value of type $C'\texttt{<}Ts_0\texttt{>}$ whose class is a subclass of C has type $C\texttt{<}Ts\texttt{>}$, and rule IFTHEN compares only class names.

4.6 Typing of Programs, Classes and Methods

$\boxed{\vdash \text{Decls } e \text{ Ok}}$

PROG-OK
$$\frac{\forall \text{Dec} \in \text{Decls}.\ \text{Dec Ok} \qquad \emptyset;\emptyset \vdash e : _}{\vdash \text{Decls } e \text{ Ok}}$$

$\boxed{\Delta;\Gamma \vdash M \text{ Ok}}$

METHOD-OK
$$\frac{\Delta, \Delta' \vdash \text{cod}(\Delta') \text{ Ok} \qquad \Delta, \Delta' \vdash T_0, \ldots, T_n \text{ Ok} \qquad \Delta, \Delta'; \Gamma, T_1\ x_1, \ldots, T_n\ x_n \vdash e : T_0}{\Delta;\Gamma \vdash \texttt{<}\Delta'\texttt{>}T_0\ m\texttt{(}T_1\ x_1, \ldots, T_n\ x_n\texttt{) \{ return } e\texttt{; \}} \text{ Ok}}$$

$\boxed{\text{Dec Ok}}$

CLASS-OK
$$\frac{\Delta \vdash \text{cod}(\Delta) \text{ Ok} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}, T_1, \ldots, T_n \text{ Ok} \qquad \Delta; C\texttt{<}\text{dom}(\Delta)\texttt{> this} \vdash Ms \text{ Ok} \qquad \text{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\texttt{class } C\texttt{<}\Delta\texttt{> extends } C'\texttt{<}Ts\texttt{> \{ }Ms\ T_1\ f_1\texttt{; }\ldots T_n\ f_n\texttt{; \}} \text{ Ok}}$$

Define $\text{overrideOk}(\Delta, T, Ms)$ $\quad$ $\text{overrideOk}(T, M)$

$\text{overrideOk}(\Delta, T, Ms)$ iff $\text{overrideOk}(C\texttt{<}Ts\texttt{>}, M)$ for every $M \in Ms$ and every $C\texttt{<}Ts\texttt{>}$ such that $\Delta \vdash T <: C\texttt{<}Ts\texttt{>}$

$\text{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<_>_}\ m\texttt{(_) \{ _ \}})$ iff $m \notin Ms$
where `class` C`<_> extends _<_> {` Ms `_ }` declared

$\text{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1, \ldots, T_n'\ x_n\texttt{) \{ _ \}})$
where `class` $C\texttt{<}\Delta\texttt{>}$ `extends _<_> {` Ms `_ }` declared
$\texttt{<}\Delta''\texttt{>}T_0\ m\texttt{(}T_1\ x_1', \ldots, T_n\ x_n'\texttt{) \{ _ \}} \in Ms$
$\text{cod}(\Delta'')[\Delta, \Delta'' = Ts, \text{dom}(\Delta')] = \text{cod}(\Delta')$
$T_0, \ldots, T_n[\Delta, \Delta'' = Ts, \text{dom}(\Delta')] = T_0', \ldots, T_n'$

$\text{overrideOk}(C\texttt{<}Ts\texttt{>}, \texttt{<}\Delta'\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1, \ldots, T_n'\ x_n\texttt{) \{ _ \}})$
where `new` $C\texttt{<}Fs\texttt{>}$ `extends _<_>(_){`Ms`}` declared
$\texttt{<}\Delta''\texttt{>}T_0\ m\texttt{(}T_1\ x_1', \ldots, T_n\ x_n'\texttt{) \{ _ \}} \in Ms$
$\text{cod}(\Delta'')[Fs, \Delta'' = Ts, \text{dom}(\Delta')] = \text{cod}(\Delta')$
$T_0, \ldots, T_n[Fs, \Delta'' = Ts, \text{dom}(\Delta')] = T_0', \ldots, T_n'$

Rule CLASS-OK types the methods of a top-level class with `this` of type $C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}$. A method that a class declares again must keep the signature it has in every superclass, up to the names of its own type variables: CALL reads the signature from the class chosen by subsumption, while the reduction runs the method of the class of the receiver. The last case of $\mathsf{overrideOk}(T, M)$ compares a method of a local class with the method of the same name in the declaration of the local class, in the same way.

4.7 Kinding and Subtyping

$$\boxed{\Delta \vdash T \ \mathrm{Ok}}$$

$$\text{WF-VAR}\ \frac{X\ \texttt{extends}\ _ \in \Delta}{\Delta \vdash X\ \mathrm{Ok}} \qquad \text{WF-CLASS}\ \frac{\mathsf{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0 \qquad \Delta \vdash Ts\ \mathrm{Ok} \qquad \Delta \vdash Ts_0 <: \mathrm{cod}(\Delta_0)[\Delta_0 = Ts_0]}{\Delta \vdash C\texttt{<}Ts\texttt{>}\ \mathrm{Ok}}$$

$$\boxed{\Delta \vdash T <: T'}$$

$$\text{SUB-REFL}\ \frac{}{\Delta \vdash T <: T} \qquad \text{SUB-INHERIT}\ \frac{\texttt{class}\ C\texttt{<}\Delta_0\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \texttt{\{}\ _\ _\ \texttt{\}}\ \mathsf{declared} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}[\Delta_0 = Ts_0] <: T'}{\Delta \vdash C\texttt{<}Ts_0\texttt{>} <: T'}$$

$$\text{SUB-INHERIT-LOCAL}\ \frac{\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>(_)\{_\}}\ \mathsf{declared} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[Fs = Ts] <: T'}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T'} \qquad \text{SUB-VAR}\ \frac{X\ \texttt{extends}\ T \in \Delta \qquad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}$$

Rule WF-CLASS kinds both top-level and local classes, including a local class written in a method of another local class. Every type in Ts_0 is checked against the bound of its type variable in Δ_0. A funnelled type variable that is not in scope where its local class C is written leaves $\mathsf{scope}(C\texttt{<}Ts\texttt{>})$ undefined; thus no type $C\texttt{<}Ts\texttt{>}$ is well kinded.

4.8 Substitution of Type Variables

Define $e[Xs = Ts]$ $M[Xs = Ts]$ $F[Xs = Ts]$ $\Delta[Xs = Ts]$ $T[Xs = Ts]$

$$
\begin{array}{lcl}
x[Xs = Ts] & = & x \\
e.\texttt{<}Ts'\texttt{>}m(es')[Xs = Ts] & = & e[Xs = Ts].\texttt{<}Ts'[Xs = Ts]\texttt{>}m(es'[Xs = Ts]) \\
e.\texttt{[}T'\texttt{]<}Ts''\texttt{>}m(es')[Xs = Ts] & = & e[Xs = Ts].\texttt{[}T'[Xs = Ts]\texttt{]<}Ts''[Xs = Ts]\texttt{>} \\
 & & m(es'[Xs = Ts]) \\
e.f[Xs = Ts] & = & e[Xs = Ts].f \\
\texttt{new } C\texttt{<}Ts'\texttt{>(}es\texttt{)}[Xs = Ts] & = & \texttt{new } C\texttt{<}Ts'[Xs = Ts]\texttt{>(}es[Xs = Ts]\texttt{)} \\
\texttt{if (}e_0\texttt{ instanceof } T'\ x\texttt{)} & = & \texttt{if (}e_0[Xs = Ts]\texttt{ instanceof } T'[Xs = Ts]\ x\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [Xs = Ts] & & \texttt{\{ }e_1[Xs = Ts]\texttt{ \} else \{ }e_2[Xs = Ts]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts'\texttt{>} & = & \texttt{new } C\texttt{<}Fs[Xs = Ts]\texttt{> extends } C'\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [Xs = Ts] & & \texttt{(}es[Xs = Ts]\texttt{)\{}Ms[Xs = Ts]\texttt{\}} \\
X'\texttt{ funnel}[Xs = Ts] & = & X'\texttt{ funnel} \quad \text{if } X' \notin Xs \\
X_i\texttt{ funnel}[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i \\
G[Xs = Ts] & = & G \\
\texttt{<}\Delta\texttt{>}T_0'\ m\texttt{(}T_1'\ x_1, \ldots, T_k'\ x_k\texttt{)} & = & \texttt{<}\Delta[Xs = Ts]\texttt{>}T_0'[Xs = Ts]\ m\texttt{(} \\
\texttt{\{ return } e\texttt{; \}}\ [Xs = Ts] & & \quad T_1'[Xs = Ts]\ x_1, \ldots, T_k'[Xs = Ts]\ x_k\texttt{)} \\
 & & \texttt{\{ return } e[Xs = Ts]\texttt{; \}} \\
(X'\texttt{ extends } T')[Xs = Ts] & = & X'\texttt{ extends } T'[Xs = Ts] \\
C\texttt{<}Ts'\texttt{>}[Xs = Ts] & = & C\texttt{<}Ts'[Xs = Ts]\texttt{>} \\
X'[Xs = Ts] & = & X' \quad \text{if } X' \notin Xs \\
X_i[X_1, \ldots, X_n = T_1, \ldots, T_n] & = & T_i
\end{array}
$$

Substitution replaces every occurrence of a type variable, *including* the ones in the method signatures of a method-local class. Substitution does not rename the type variables it meets. Substitution of type variables into expressions happens only during reduction, where the substituted types are ground. No renaming is needed: the substituted types are ground, and the domain of the substitution never contains a type variable bound inside the substituted term, by requirement A7. In the domain of a substitution, bounds Δ stand for $\mathrm{dom}(\Delta)$, and funnels Fs stand for their funnelled type variables: $T[F_1, \ldots, F_n = T_1, \ldots, T_n]$ replaces X_i with T_i for every $i \in 1..n$ such that $F_i = X_i$ `funnel`; the ground funnels are ignored. Thus X_i `funnel`$[X_1, \ldots, X_n = T_1, \ldots, T_n] = T_i$ is a ground type G, and the result is again a funnel.

4.9 Substitution of Variables

Define $e[xs = vs]$ $M[xs = vs]$

$$
\begin{array}{lcl}
x[xs = vs] & = & x \quad \text{if } x \notin xs \\
x_i[x_1, \ldots, x_n = v_1, \ldots, v_n] & = & v_i \\
e.\texttt{<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{<}Ts\texttt{>}m(es'[xs = vs]) \\
e.\texttt{[}T\texttt{]<}Ts\texttt{>}m(es')[xs = vs] & = & e[xs = vs].\texttt{[}T\texttt{]<}Ts\texttt{>}m(es'[xs = vs]) \\
e.f[xs = vs] & = & e[xs = vs].f \\
\texttt{new } C\texttt{<}Ts\texttt{>(}es\texttt{)}[xs = vs] & = & \texttt{new } C\texttt{<}Ts\texttt{>(}es[xs = vs]\texttt{)} \\
\texttt{if (}e_0\texttt{ instanceof } T\ x'\texttt{)} & = & \texttt{if (}e_0[xs = vs]\texttt{ instanceof } T\ x'\texttt{)} \\
\texttt{\{ }e_1\texttt{ \} else \{ }e_2\texttt{ \}}\ [xs = vs] & & \texttt{\{ }e_1[xs = vs]\texttt{ \} else \{ }e_2[xs = vs]\texttt{ \}} \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs = vs] & & \texttt{(}es[xs = vs]\texttt{)\{}Ms[xs = vs]\texttt{\}} \\
 & & \quad \text{if } \texttt{this} \notin xs \\
\texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} & = & \texttt{new } C\texttt{<}Fs\texttt{> extends } C'\texttt{<}Ts\texttt{>} \\
\texttt{(}es\texttt{)\{}Ms\texttt{\}}\ [xs, \texttt{this}, xs' = vs, v, vs'] & & \texttt{(}es[xs, \texttt{this}, xs' = vs, v, vs']\texttt{)\{}Ms[xs, xs' = vs, vs']\texttt{\}} \\
\texttt{<}\Delta\texttt{>}T_0\ m\texttt{(}T_1\ x_1', \ldots, T_k\ x_k'\texttt{)} & = & \texttt{<}\Delta\texttt{>}T_0\ m\texttt{(}T_1\ x_1', \ldots, T_k\ x_k'\texttt{)} \\
\texttt{\{ return } e\texttt{; \}}\ [xs = vs] & & \texttt{\{ return } e[xs = vs]\texttt{; \}}
\end{array}
$$

A method-local class binds `this`, as in Java: inside the methods of a method-local class, `this` is the local object, and the outer `this` is hidden. Thus substitution of variables enters the methods of a method-local class for every variable except `this`. The variable x' of an `if` and the parameters $x'_1, \ldots, x'_k$ of a method are never in xs, by requirement A7; so no renaming is needed.

...

4.10 Java Typing of Local Classes

Rule NEW-LOCAL-JAVA types local classes as Java does. A local class has no funnels, and its methods are typed under the full Δ and Γ in scope, except for `this`. Replacing rule NEW-LOCAL with rule NEW-LOCAL-JAVA makes our system unsound for the same reason Java is unsound. Consider the method `mk` below, the method `m` of Example 1 in our syntax:

```
<X> Foo<X> mk(Object<> other, X x){ return
  if (other instanceof MyFoo<> w){ w }
  else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; }
```

Rule NEW-LOCAL-JAVA gives to the object created by `mk` the type `MyFoo<>`. Rule SUB-INHERIT-LOCAL reads the declaration of `MyFoo`, so `MyFoo<>` is a subtype of `Foo<X>`, where `X` is the type variable of `mk`. The type `MyFoo<>` does not say which type `X` stood for when the object was created. Since generic(`MyFoo`) = `MyFoo<>` has no type variables, rule CAST-DOWN accepts the type test `other instanceof MyFoo<>` from `Object<>`. Thus the call `a.<String>mk(a.<Integer>mk(o, i), s)` has type `Foo<String>`, but it returns a `Foo<Integer>`. This is the unsoundness of Java. It is there even if we keep the restriction of Java that the name of a local class can only be written inside the method declaring it, as `mk` does. Our formal system shows two more failure points. They are the same failure, shown more explicitly:

- Reduction does not preserve types. The call `a.<Integer>mk(o, i)` has type `Foo<Integer>`. The value `o` is not a `MyFoo`, so the type test fails and the call takes the `else` branch. It reduces to the value

  ```
  new MyFoo<> extends Foo<Integer>(){ Integer x(){ return i; } }
  ```

  The type of this value is `MyFoo<>`, and `MyFoo<>` is not a subtype of `Foo<Integer>`. The supertypes of `MyFoo<>` come from the declaration of `MyFoo` in `mk`, not from the value: rule SUB-INHERIT-LOCAL derives $\emptyset \vdash$ `MyFoo<>` $<:$ `Foo<X>`, where `X` is the type variable of `mk`. Subtyping does not check kinding, so this judgement holds in the empty bounds $\emptyset$, where `X` is not in scope. However, `Foo<X>` is not well kinded under $\emptyset$, so rule SUBS cannot give the value the type `Foo<X>` either. Java has no typing of values during reduction, so in Java this failure is invisible.
- Typing depends on the names of type variables. Our system lets the name of a local class be written anywhere. Consider the methods

  ```
  <Y> MyFoo2<> mk2(Y y){
    return new MyFoo2<> extends Foo<Y>(){ Y x(){ return y; } }; }
  <Y> Foo<Y> leak(MyFoo2<> z){ return z; }
  ```

  The method `leak` is well typed, since the type variable `Y` of `leak` has the same name as the type variable `Y` of `mk2`. Renaming `Y` into `Z` in `leak` makes `leak` ill typed. The call

  ```
  a.<String>leak(a.<Integer>mk2(i))
  ```

has type `Foo<String>`, but it returns a `Foo<Integer>`, without type tests.

Funnelling solves the problem in all three forms. Under rule NEW-LOCAL, the object created by `mk` has type `MyFoo<X>`, written `new MyFoo<X funnel> extends Foo<X>(){..}`. Since generic(`MyFoo`) = `MyFoo<X>`, rule CAST-DOWN accepts the type test `instanceof MyFoo<X>` from `Foo<X>`, but not from `Object<>`. Thus the funnelled `mk` takes `Foo<X> other` instead of `Object<> other`, and tests `other instanceof MyFoo<X> w`. The call `a.<Integer>mk(o, i)`, with `o` of type `Foo<Integer>`, reduces to a value of type `MyFoo<Integer>`, a subtype of `Foo<Integer>`.

...

briefly note that Java allows extra, "fresh" generic parameters in local classes

call out that we're allowed to declare return types that mention an inner class; inner classes are "public", and our approach works even in this case, the more restrictive case of Java is naturally covered

articulate early: It's not just about generic reification: just checking type parameters aren't enough, as our MyFoo cases have no generic params. You need to *deeply* check reified generics, you need to check not just the base type but also check the choice(s) of parent types from the context into the runtime check.

5 (Un)Soundness of Method Local Classes

With our language primitives in place, we now consider type safety of two variants of our language. The two variants differ only in the typing of local classes:

1. ζ(Java) types local classes with rule NEW-LOCAL-JAVA, as Java does;
2. ζ(funnelled) types local classes with rule NEW-LOCAL.

Both variants use the type test of Java: rule IF-INSTANCEOF accepts only checked casts, and at run time rules IFTHEN and IFELSE compare class names only. As shown in Section 4, ζ(Java) is unsound. We now show that ζ(funnelled) is sound.

▶ **Theorem 1** (Progress). *Under a global* Decls *such that* Dec Ok *for every* Dec $\in$ Decls*: if* $\emptyset;\emptyset \vdash e : T$*, then either* e *is of form* v*, or there is an* e' *such that* $e \to e'$*.*

▶ **Theorem 2** (Preservation). *Under a global* Decls *such that* Dec Ok *for every* Dec $\in$ Decls*: if* $\emptyset;\emptyset \vdash e : T$ *and* $e \to e'$*, then* $\emptyset;\emptyset \vdash e' : T$*.*

▶ **Corollary 3** (Type Soundness). *Under a global* Decls *such that* Dec Ok *for every* Dec $\in$ Decls*: if* $\emptyset;\emptyset \vdash e_0 : T$*, and* $e_0 \to^* e$*, then* $\emptyset;\emptyset \vdash e : T$ *and either* e *is of form* v*, or there is an* e' *such that* $e \to e'$*.*

6 Related Work

It is not uncommon to have "intentional unsoundness" in industrial languages, such as Java's covariant array subtyping, or TypeScript's covariant method subtyping. However these are intentionally designed into the language to improve ease of use, and in most cases (like Java's array handling) problematic uses are detected at runtime (e.g., Java checks upon storing into an array that the new entry value is a subtype of the array's original content type).

However, unintentional unsoundness in industrial type systems is rarer. The most notable example is Amin and Tate's discovery [1] that Java and Scala's handling of generic type constraints was rendered unsound by the fact that `null` is treated as inhabiting all reference types, making all type constraints (even nonsensical ones) satisfiable. Servetto et al. [4]

first pointed out the issue publicly, though in the Q&A session after the talk, multiple Java designers were unsurprised — as Gilad Bracha put it, "Once I saw the title of your talk, I understood what the problem was."

Colin says: As I listened to the Q&A session again, James framed this as a question of whether two instances allocated in different contexts should be considered the same or not. I think that might be useful framing to bring up early here. Gilad also pointed out that (paraphrasing) the BETA people knew an inner class was a property of the instance, not its own class.

discuss C# casts with reified generics

Flix is a functional language for the JVM. The Flix compiler monomorphizes polymorphic functions, as Rust and MLton do. Flix has no subtyping. The Flix construct closest to a local class is the `new` expression, which creates an object of an unnamed class that extends a Java class or implements a Java interface . A Flix program can not name such a class, so a Flix program can not write a type test against it, as with the generated classes of C#. Thus the type test of Example 1 can not be written in Flix.

cite Flix FAQ, for monomorphization and no subtyping

cite Flix anonymous classes

The key issues at work in the interaction of method-local inner classes and generics arise from the interaction of (1) nominal dynamic downcasts, (2) design choices about what runtime information about generic parameters is both persisted *and checked* by casts, and (3) (not) tracking all dependencies on type parameters in the type system. Essentially, Java and Scala both treat `MyFoo` as a sort of existential type — that there exists some `X` such that `MyFoo` extends `Foo<X>`. However, treating `MyFoo` as a closed rather than generic type is equivalent to assuming that there is only one existential for all instances — associated with the declaration site — rather than potentially different existentials for different instances. This combined with the fact that the JVM is unable to dynamically check for different instantiations leads to the unsoundness. Our dynamic solution resolves this with a specific implementation of casts on reified generics.

Modern Java: under modern Java there is no separate dynamic solution; see the note on the dynamic approach in the introduction.

Our static solution effectively prohibits the de facto existential capture of the enclosing type parameter.

The decision to treat types like the `MyFoo` variants in our examples as closed existential types that depend on other choices of types is not without parallels in other languages. Haskell and Rust, for example, include associated types [2], where a typeclass has a type as a member. A yet closer analogue is Scala's path-dependent types , where types may be *members* of an individual object, rather than associated with an individual class. The code below adapts the troublesome example method using Scala's type members:

cite dot

```
trait Foo { type X; def x() : X; }
def m[Y](other: Any, newx: Y) : Foo2 =
    class MyFoo extends Foo { type X = Y; override def x() : X = newx; }
    newx match {
        case f: MyFoo => f
        case _ => MyFoo()
    }
```

This code compiles without warnings — and is not an issue. In this case the dependency of `MyFoo` instances on the type parameter used to call `m` is made explicit with a type member. But Scala now treats thes

7 Conclusions

write the conclusions

8 Technical Clarifications

The well-formedness requirements A1-A11 of Section 4 are part of the grammar. A metavariable ranges only over the syntactic forms that satisfy them. For example, by requirement A4, the grammar of M does not generate a sequence of symbols with the shape of a method declaring two parameters named `x`, so no metavariable M stands for that sequence of symbols. Thus every program Decls e satisfies the well-formedness requirements.

The type rules and the reduction rules are metarules. An instance of a metarule replaces every metavariable of the metarule, consistently, with a syntactic form of the grammar of that metavariable. A judgement holds when it is the conclusion of a derivation built from instances of the metarules. In particular, an instance of a reduction rule $e \to e'$ exists only when the right side e' is an expression. Thus a reduction step never produces a sequence of symbols that the grammar does not generate. When the substitutions of a reduction rule would produce such a sequence of symbols for a redex, that reduction rule has no instance for that redex. The proof of Progress shows that, for every well-typed expression that is not a value, some reduction rule has an instance.

9 Draft Notes

Marco had this idea that casts may be avoided by using dynamic dispatch returning this. Can not work, since Meth local class is private, there is no way an external interface can mention it to provide the cast method. Of course, the method local is private to the body (this means also the method signature can not use it) This means no self:Foo method, nor self:T method of a generic class eventually instantiating T=Foo.

This means the core language DO NEED if(e instanceof T x) else

//nested funnelling: new A<X>() B<X> f()-> new B<X>() new A<X>() <K>B<X,K> f()-> new B<X,K>()

using 'extends' for declarations in top level classes and generic methods using 'funnel' for named inner classes new A<X funnel Object>() B<X> f()-> new B<X funnel Object>() new A<X funnel Object>() <K extends Object> B<X,K> f()-> new B<X funnel Object,K funnel Object>() or, if the bounds are not repeated new A<X funnel>() <K extends Object> B<X,K> f()-> new B<X funnel,K funnel>() Very funky syntax: we get new B<X funnel Object,K funnel Object>() but, during reduction, this can become we get new B<Person,K funnel Object>() where we have a mixture of concrete types and generic funnelling

Global class definition uses class C<X extends Bar>.. named method local class uses new C<X funnel>().. normal new: new C<Ts>()

```
e ::= x | e.m(es) | e.f | new C<Ts>(es) | new C<Fs>(){Ms} | if (e0 instanceof T x){ e1 }else{
T ::= C<Ts> | X
F ::= C<CCs> | X funnel// Important, can not be T | X funnel, otherwise we could have an X no
CC::= C<CCs> //types with zero free type variables
B::=  X extends T // Need T, not F, to be able to do things like (Java) class Bar<X,Y extends
Dec::= class C<Bs>{ Ms Ss} //assume standard constructor
S ::= T x; //field/slot
```

```
M ::= Sig; | Sig{ return e; }
Sig::= <Bs> T m(x1:T1,..xn:Tn)
G ::= x1:T1..xn:Tn
```

//we do not need Object to be able to write 'extends Object' //this is exactly as the distinction of checked and unchecked exceptions: formally it evaporates in syntacitc sugar where all is checked //and all methods 'throws Error, RTE' // Can assume the user provides their own Object type Note: an use of X is just X, a declaration of X is X extends T; a funnelling of X is X funnel Program is just many Dec plus an e typable in empty MARCO TODO: typechecking Decs;Bs;G|-e:T Decs|-Dec:ok Decs;T|-M:ok e –> e' Note, there are multiple semantics/ts: e instanceof CC //always clear e instanceof T with T not of form CC would not typecheck

e –> e' for Java with warn->err: just have CC in instanceof Java with magic reifined cast: just have T in instanceof (since reduction will replace(thus remember) generics Java with warn->off: AT RUN TIME instanceof only checks base class, ignores generic parameters AT COMPLILE TIME, instanceof can be err if the full type (including generics) is not potentially correlated)?? unobvious consider interfaces In prose, should explain that we won't flag cast of some type Foo to some type Bar, even though Java sometimes flags unrelated casts

Modern Java: these three semantics are replaced by the single one of Java 16+: erased run-time check, plus the checked-cast premise of rule IF-INSTANCEOF (Section 4.5). Unrelated types are rejected by it, as in Java.

<X extends List<Y funnel»
class Foo<X> class Getter<X funnel> <Y extends List<X» X getFirst(Y y) y.getOrElse(0,null)

<Y> .. return foo<Y,MyX>(); <Y,X extends List<Y» foo() return Foo<X funnel>()... X ;

A Lean Formalization

This appendix relates the formalism of Section 4 and Section 4.10 to a Lean 4 formalization in the folder `lean` of the paper repository. The Lean formalization is set up, but the proofs of the soundness theorems are not done yet.

A.1 Request

This appendix answers the following request, reported verbatim, so that we can check later how much of it is addressed.

```
The Appendix, for each piece of formalism we report the following:

(A)Original formalism

(B)Corresponding Lean formalism

(C)English explanation of the differences and either
- why it ok for those differences to be there.
- or what the more appropriate form of the original formalism should be
```

```
(D)A clear report of the lean statements and an high level description of the
    proofs.

(E)A list of validation examples. Those are:
- programs that Do pass the lean type system AND the paper type system, or that do
    not pass both the lean type system and the paper type system.
- programs that reduce in some steps equally in both the paper type system and the
    lean type system. Basically, this is 'testing': we are empirically checking
    that the two formalism are the same by checking that they behave the same on a
     range of examples including interesting corner cases.
```

A.2 Status

The file `lean/lean-toolchain` selects Lean 4.35.0: elan runs that version for every command in the folder `lean`. The project depends on Mathlib v4.35.0, which brings Batteries and Aesop; `lake exe cache get` downloads Mathlib already compiled. The current files import only the core library of Lean. `lake build` in the folder `lean` checks all the files. The formalization has two levels. The *high level*, `lean/Funnelling/High`, follows the metarules of the paper as closely as possible; Section A.4 compares it with the paper. The *low level*, `lean/Funnelling/Low`, is free: it can change in any way that makes the proofs easier, and several low levels can be tried, without changing the high level. The low level starts as a copy of the high level, with its own datatypes and functions. The files are:

- `lean/Funnelling/High/Syntax.lean`: types, funnels, expressions, methods, declarations, class declarations K.
- `lean/Funnelling/High/Substitution.lean`: substitution of type variables and of variables, and the renaming used by fresh.
- `lean/Funnelling/High/Declarations.lean`: declared, scope, the pairing, generic, $M \in C$, typeOf, values, fresh.
- `lean/Funnelling/High/WellFormed.lean`: the well-formedness requirements A1-A11.
- `lean/Funnelling/High/Typing.lean`: subtyping, kinding, checked casts, overrideOk, consistent, typing, method, class and program typing.
- `lean/Funnelling/High/Reduction.lean`: evaluation contexts and the reduction.
- `lean/Funnelling/Low/*.lean`: the same files for the low level, and `lean/Funnelling/Low/Soundness.lean`, progress and preservation of the low level.
- `lean/Funnelling/Translation.lean`: the translation from the high level to the low level.
- `lean/Funnelling/Adequacy.lean`: the lemmas that relate the two levels through the translation.
- `lean/Funnelling/Soundness.lean`: the soundness statements of the paper, on the high level.
- `lean/Funnelling/Examples.lean`: the validation examples, on the high level.

Lean accepts all the files. The high-level `progress` and `preservation` are proved from the low-level ones through the adequacy lemmas. The high-level `typeSoundness` is proved from `progress`, `preservation` and `stepWellFormed`. The low-level `progress` and `preservation`, the six adequacy lemmas and `stepWellFormed` are stated with `sorry`. Section A.6 lists which validation examples are proved and which are stated with `sorry`. The mini proof that checked the installation is not in the repository: it proved $a + b = b + a$ with `omega`, Even 4 for an inductive predicate Even, ¬Even 1 by `cases`, and evaluated an expression with `#eval`.

The Lean code is written in ASCII: `->` is $\to$, `/\` is $\wedge$, `\/` is $\vee$, `List.Mem x xs` is $x \in xs$, and `delta`, `gamma`, `sigma` are Δ, Γ, σ.

A.3 Differences Common to All the Pieces

The following differences hold for every piece below, and the pieces do not repeat them.

- **Names.** Lean has one type `Name` (strings) for class names, method names, field names, variables and type variables. The paper uses a different metavariable for each. This is fine: no definition compares two names of different kinds.
- **Grammar and well-formedness.** In the paper, the metavariables range over the terms of the grammar, and the well-formedness requirements restrict the grammar (Section 8). In particular, a metavariable v ranges over values and a metavariable G ranges over ground types. Lean defines the raw syntax as inductive types, and the restrictions as Boolean functions: `Expr.isValue` for values, `Ty.isGround` for ground types, `wfDecls` and `wfExpr` for A1-A11. Thus every Lean rule with a metavariable v or G of the paper has an explicit premise `isValue` or `allGround`. The theorems assume `wfDecls` and `wfExpr` for the initial program. The Lean reduction does not require the result to be well formed; the theorem `stepWellFormed` states that it is. In the paper an instance of a reduction rule exists only when its result is an expression, so the paper needs no such lemma. The two formulations agree when `stepWellFormed` holds.
- **Sequences.** The paper applies a judgement to sequences position by position, and requires the same length (Section 4.3). Lean writes a list premise as a length equality and a premise for every pair in `List.zip`, or as a premise for every element of a list.
- **Substitution and lengths.** The paper defines $T[\Delta = Ts]$ only when Δ and Ts have the same length. Lean builds a substitution with `mkSubst xs ts`, which is `none` when the lengths differ; a premise `mkSubst xs ts = some sigma` requires the same length.
- **Functions instead of relations.** The paper defines declared, scope, generic, the pairing and typeOf with **Define** boxes; those definitions have at most one result. Lean defines them as functions, so the validation examples can compute them. The inference rules (reduction, typing, kinding, subtyping, checked cast, method typing) are inductive relations in Lean, as in the paper.
- **Equality of types.** Lean derives no decidable equality for the nested inductive types; `Ty.beq` is a Boolean equality on types, used only by the pairing. The proofs need the lemma `Ty.beq t t' = true <-> t = t'`.

A.4 Side by Side: Paper and Lean

Each block below shows, on the left, a piece of the formalism of the paper, with the minimal text of the paper that the piece needs; on the right, the Lean definitions of the same concepts in the high level `Funnelling.High`; under them, the words that map one side to the other, and the differences beyond Section A.3.

A.4.1 Syntax

Types, funnels, bounds and type environments

$$
\begin{array}{lcl}
T & ::= & C\texttt{<}Ts\texttt{>} \mid X \\
F & ::= & G \mid X\ \texttt{funnel} \\
G & ::= & C\texttt{<}Gs\texttt{>} \\
\Gamma & ::= & T_1\ x_1, \ldots, T_n\ x_n \\
\Delta & ::= & X_1\ \texttt{extends}\ T_1, \ldots, X_n\ \texttt{extends}\ T_n
\end{array}
$$

A ground type G is a type with no type variables. For a type variable X, $\Delta(X)$ is the bound of X in Δ.

```
inductive Ty where
  | cls (c : Name) (args : List Ty)
  | var (x : Name)
inductive Funnel where
  | ground (g : Ty)
  | funnel (x : Name)
abbrev Bounds := List (Prod Name Ty)
abbrev Ctx := List (Prod Name Ty)
def Ty.isGround : Ty -> Bool
  | .var _ => false
  | .cls _ ts => Ty.allGround ts
```

Mapping. G is a `Ty` with `isGround`. A pair of Γ is `(x, T)`: name first. $\Delta(X)$ is `delta.lookup X` and $\Gamma(x)$ is `gamma.lookup x`.

Expressions and methods

$$
\begin{array}{lcl}
e & ::= & x \mid e\texttt{.<}Ts\texttt{>}m(es) \mid e\texttt{.[}T\texttt{]<}Ts\texttt{>}m(es) \\
 & & \mid e.f \mid \texttt{new}\ C\texttt{<}Ts\texttt{>}(es) \\
 & & \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts'\texttt{>}(es)\{Ms\} \\
 & & \mid \texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \\
M & ::= & \texttt{<}\Delta\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e;\ \}
\end{array}
$$

```
mutual
inductive Expr where
  | var (x : Name)
  | call (recv : Expr) (targs : List Ty) (m : Name) (
     args : List Expr)
  | superCall (recv : Expr) (start : Ty) (targs : List
     Ty) (m : Name) (args : List Expr)
  | field (recv : Expr) (f : Name)
  | new (c : Name) (targs : List Ty) (args : List Expr)
  | newLocal (c : Name) (fs : List Funnel) (sup : Name)
      (supArgs : List Ty)
     (args : List Expr) (ms : List Method)
  | ifInst (e0 : Expr) (t : Ty) (x : Name) (e1 e2 :
     Expr)
inductive Method where
  | mk (bounds : Bounds) (ret : Ty) (m : Name) (params
     : List (Prod Ty Name)) (body : Expr)
end
```

Mapping. The super call $e\texttt{.[}T\texttt{]<}Ts\texttt{>}m(es)$ is `superCall`; the local class is `newLocal`; the type test is `ifInst`. A parameter $T_i\ x_i$ is the pair `(T, x)`.

Declarations and programs

$$
\begin{array}{lcl}
\text{Dec} & ::= & \texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ Ss\ \} \\
S & ::= & T\ f; \\
K & ::= & \text{Dec} \mid \texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\}
\end{array}
$$

A program is a class table Decls and a main expression e.

```
structure Dec where
  name : Name
  bounds : Bounds
  sup : Name
  supArgs : List Ty
  methods : List Method
  fields : List (Prod Ty Name)
inductive K where
  | top (dec : Dec)
  | loc (c : Name) (fs : List Funnel) (sup : Name) (
     supArgs : List Ty)
      (args : List Expr) (ms : List Method)
abbrev Decls := List Dec
```

Mapping. The slots Ss are `fields`, pairs `(T, f)`. A program is a pair `(d : Decls)` and `(e : Expr)`, passed separately.

Values

$$
v ::= \texttt{new}\ C\texttt{<}Gs\texttt{>}(vs) \mid \texttt{new}\ C\texttt{<}Gs\texttt{>}\ \texttt{extends}\ C'\texttt{<}Gs'\texttt{>}(vs)\{Ms\}
$$

```
def Expr.isValue : Expr -> Bool
  | .new _ gs vs => Ty.allGround gs && Expr.allValues
     vs
  | .newLocal _ fs _ gs vs _ =>
    fs.all Funnel.isGround && Ty.allGround gs && Expr.
       allValues vs
  | _ => false
```

Mapping. The funnels Gs of a value of a local class are ground funnels: `fs.all Funnel.isGround`.

Evaluation contexts

$$
\begin{array}{rcl}
\mathcal{E} & ::= & \square \mid \mathcal{E}.\texttt{<}Gs\texttt{>}m(es) \mid v.\texttt{<}Gs\texttt{>}m(vs, \mathcal{E}, es) \\
& & \mid \mathcal{E}.[G]\texttt{<}Gs\texttt{>}m(es) \\
& & \mid v.[G]\texttt{<}Gs\texttt{>}m(vs, \mathcal{E}, es) \\
& & \mid \mathcal{E}.f \mid \texttt{new } C\texttt{<}Gs\texttt{>}(vs, \mathcal{E}, es) \\
& & \mid \texttt{new } C\texttt{<}Gs\texttt{> extends } C'\texttt{<}Gs'\texttt{>}(vs, \mathcal{E}, es)\{Ms\} \\
& & \mid \texttt{if } (\mathcal{E} \texttt{ instanceof } G\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \}
\end{array}
$$

```
inductive ECtx where
  | hole
  | callRecv (ctx : ECtx) (gs : List Ty) (m : Name) (es
      : List Expr)
  | callArg (v : Expr) (gs : List Ty) (m : Name) (vs :
     List Expr) (ctx : ECtx) (es : List Expr)
  | superRecv (ctx : ECtx) (g : Ty) (gs : List Ty) (m :
      Name) (es : List Expr)
  | superArg (v : Expr) (g : Ty) (gs : List Ty) (m :
     Name) (vs : List Expr) (ctx : ECtx) (es : List
     Expr)
  | field (ctx : ECtx) (f : Name)
  | newArg (c : Name) (gs : List Ty) (vs : List Expr) (
     ctx : ECtx) (es : List Expr)
  | localArg (c : Name) (fs : List Funnel) (c' : Name)
     (gs : List Ty) (vs : List Expr) (ctx : ECtx)
     (es : List Expr) (ms : List Method)
  | ifCond (ctx : ECtx) (g : Ty) (x : Name) (e1 e2 :
     Expr)
```

Mapping. $\mathcal{E}[e]$ is `ECtx.plug E e`. `ECtx.ok E` checks that the v are values and the G are ground.

A.4.2 Well-Formedness

A1, A2, A3

(A1) In Decls, class names are unique across top-level and local classes.
(A2) In a class, all the field names, including the field names of the superclasses, are disjoint.
(A3) In a class, top-level or local, all the method names are disjoint.

```
def a1 (d : Decls) : Bool := nodup ((declaredKs d).map
   K.name)
def a2 (d : Decls) : Bool :=
  d.all fun dec => nodup ((superChain d (d.length + 1)
     dec.name).flatMap
    fun c => c.fields.map Prod.snd)
def a3 (ks : List K) : Bool := ks.all fun k => nodup (k
   .methods.map Method.name)
```

Mapping. `superChain d n c` is the chain of superclasses of c, cut after n classes; an acyclic chain is never longer than |Decls|, and A9 rejects a cyclic one. A3 is checked on `declaredKs d` and on the local classes of the main expression.

A4, A5, A6

(A4) In a method, all the parameter names $x_1, \ldots, x_n$ are disjoint.
(A5) In a Δ, all the X in dom(Δ) are disjoint.
(A6) In a list of funnels Fs, all the X in dom(Fs) are disjoint.

```
def a4 (ms : List Method) : Bool := ms.all fun m =>
   nodup m.params
def a5 (d : Decls) (ms : List Method) : Bool :=
  d.all (fun dec => nodup (dom dec.bounds)) && ms.all
     fun m => nodup (dom m.bounds)
def a6 (ks : List K) : Bool := ks.all fun k => nodup (
   domF k.funnels)
```

Mapping. Each check runs on all the methods and local classes of Decls, and of the main expression.

A7

(A7) There is no shadowing of X or x, except for `this`.

A method body of a local class also binds `this`, thus shadowing the `this` of the enclosing method; this is the only form of shadowing.

```
def Expr.noShadow (tvs vs : List Name) : Expr -> Bool
  ...
  | .newLocal _ _ _ _ es ms =>
    Expr.noShadowList tvs vs es &&
    Method.noShadowList tvs (vs.filter fun x => x !=
       thisName) ms
  | .ifInst e0 _ x e1 e2 =>
    e0.noShadow tvs vs && !(vs.contains x) &&
    e1.noShadow tvs (x :: vs) && e2.noShadow tvs vs
def Method.noShadow (tvs vs : List Name) : Method ->
   Bool
  | .mk delta _ _ ps e =>
    (dom delta).all (fun x => !(tvs.contains x)) &&
    (ps.map Prod.snd).all (fun x => !(vs.contains x))
       &&
    e.noShadow (tvs ++ dom delta) (thisName :: vs ++ ps.
       map Prod.snd)
def a7Decls (d : Decls) : Bool :=
  d.all fun dec => Method.noShadowList (dom dec.bounds)
      [] dec.methods
```

Mapping. `tvs` and `vs` are the type variables and the variables in scope. The cases written `...` go into the subexpressions with the same `tvs` and `vs`. Funnels bind nothing: X `funnel` refers to the X in scope.

A8, A9

(A8) No parameter and no variable of an `if` is `this`.
(A9) Decls contains `class Object<> extends Object<> { }`, and the chain of superclasses of every other top-level class reaches `Object`.

```
def a8 (ms : List Method) (ifVars : List Name) : Bool
   :=
  ms.all (fun m => !(m.params.contains thisName)) && !(
     ifVars.contains thisName)
def a9 (d : Decls) : Bool :=
  (d.any fun dec => dec.name == objectName && dec.
     bounds.isEmpty &&
    dec.sup == objectName && dec.supArgs.isEmpty && dec.
       methods.isEmpty &&
    dec.fields.isEmpty) &&
  d.all fun dec => reachesObject d dec.name
```

Mapping. `ifVars` lists the variables of all the `if` in the program.

A10, A11

(A10) In Decls, the superclass of every class, top-level or local, is a top-level class.
(A11) In Decls, every funnel is of the form X `funnel`.

```
def a10 (d : Decls) : Bool := (declaredKs d).all fun k
   => (findDec d k.sup).isSome
def a11 (d : Decls) : Bool :=
  (declaredKs d).all fun k => k.funnels.all fun
    | .funnel _ => true
    | .ground _ => false
def groundFunnels (ks : List K) : Bool :=
  ks.all fun k => k.funnels.all fun
    | .funnel _ => true
    | .ground g => g.isGround
```

Mapping. `groundFunnels` is the grammar $G ::= C\texttt{<}Gs\texttt{>}$ of a ground funnel, not a requirement of the paper.

All the requirements

We require the well-formedness conditions A1-A11.

The well-formedness requirements A1-A11 are part of the grammar (Section 8).

```
def wfDecls (d : Decls) : Bool :=
  a1 d && a2 d && a3 (declaredKs d) && a4 d.methods &&
     a5 d d.methods &&
  a6 (declaredKs d) && a7Decls d &&
  a8 d.methods (d.flatMap fun dec => Method.ifVarsList
     dec.methods) &&
  a9 d && a10 d && a11 d && groundFunnels (declaredKs d
     )
def wfExpr (e : Expr) : Bool :=
  a3 e.locals && a4 e.methods && a5 [] e.methods && a6
     e.locals &&
  e.noShadow [] [] && a8 e.methods e.ifVars &&
     groundFunnels e.locals
```

Mapping. `wfDecls` checks Decls; `wfExpr` checks the main expression, which A3-A8 also restrict. The theorems assume both (Section A.3).

827 ### A.4.3 Declarations

K declared

Dec declared iff Dec ∈ Decls

`new` C_0`<`Fs`>` `extends` C'`<`Ts`>(`es`){`Ms`}` declared
  where K declared
    `<_>_` m`(_) { return` e`; }` ∈ K
    `new` C_0`<`Fs`>` `extends` C'`<`Ts`>(`es`){`Ms`}` occurs in e

An expression e' occurs in e when $e = \mathcal{C}[e']$, for an expression context $\mathcal{C}$. Expression contexts $\mathcal{C}$ do not enter the methods of a local class.

828

```
def Expr.locals : Expr -> List K
  | .var _ => []
  | .call e _ _ es => e.locals ++ Expr.localsList es
  | .superCall e _ _ _ es => e.locals ++ Expr.
     localsList es
  | .field e _ => e.locals
  | .new _ _ es => Expr.localsList es
  | .newLocal c fs c' ts es ms =>
    K.loc c fs c' ts es ms :: (Expr.localsList es ++
       Method.localsList ms)
  | .ifInst e0 _ _ e1 e2 => e0.locals ++ e1.locals ++
     e2.locals
def declaredKs (d : Decls) : List K :=
  d.flatMap fun dec => K.top dec :: Method.localsList
     dec.methods
def Declared (d : Decls) (k : K) : Prop := List.Mem k (
   declaredKs d)
```

Mapping. `Expr.locals` also enters the methods of the local classes it finds, so `declaredKs` lists in one pass all the local classes that the paper reaches through a chain of declared.

---

scope(C`<`Ts`>`)

scope(C`<`Ts`>`) = $\Delta; Ts$
  where `class` C`<`Δ`>` `extends` `_<_>` `{ _ _ }` declared
scope(C_0`<`Ts`>`) = $Fs \overset{\Delta,\Delta'}{\Longleftrightarrow} Ts$
  where `<`Δ'`>_` m`(_) { return` e`; }` ∈ C
    `new` C_0`<`Fs`>` `extends` `_<_>(_){_}` occurs in e
    scope(generic(C)) = $\Delta; _$

```
def Expr.scopeEntries (env : Bounds) : Expr -> List
   ScopeEntry
  ...
  | .newLocal c fs _ _ es ms =>
    (c, (env, fs)) :: (Expr.scopeEntriesList env es ++
      match pairing env fs (fs.map Funnel.abs) with
      | some (inner, _) => Method.scopeEntriesList
         inner ms
      | none => [])
def Method.scopeEntries (env : Bounds) : Method -> List
    ScopeEntry
  | .mk delta _ _ _ e => e.scopeEntries (env ++ delta)
def scope (d : Decls) (c : Name) (ts : List Ty) :
   Option (Prod Bounds (List Ty)) :=
  match findDec d c with
  | some dec => some (dec.bounds, ts)
  | none =>
    match (d.flatMap fun dec => Method.scopeEntriesList
        dec.bounds dec.methods).lookup c with
    | some (env, fs) => pairing env fs ts
    | none => none
```

829

Mapping. `env` is Δ, Δ': the bounds in scope where the local class is written. In the methods of a local class, `inner` is the bounds of scope(generic(C)). By A1, `lookup` finds the only entry of C_0.

Pairing

$\emptyset \overset{\Delta}{\Longleftrightarrow} \emptyset = \emptyset; \emptyset$
X `funnel`$, Fs \overset{\Delta}{\Longleftrightarrow} T, Ts = (X$ `extends` $\Delta(X); T) \cdot (Fs \overset{\Delta}{\Longleftrightarrow} Ts)$
$G, Fs \overset{\Delta}{\Longleftrightarrow} G, Ts = Fs \overset{\Delta}{\Longleftrightarrow} Ts$
$(\Delta; Ts) \cdot (\Delta'; Ts') = \Delta, \Delta'; Ts, Ts'$

830

```
def pairing (delta : Bounds) : List Funnel -> List Ty
   -> Option (Prod Bounds (List Ty))
  | [], [] => some ([], [])
  | .funnel x :: fs, t :: ts => do
    let b <- delta.lookup x
    let (delta', ts') <- pairing delta fs ts
    pure ((x, b) :: delta', t :: ts')
  | .ground g :: fs, t :: ts => if g.beq t then pairing
      delta fs ts else none
  | _, _ => none
```

Mapping. An undefined pairing is `none`. The operator · is the cons of the recursive case.

$M \in K$, $M \in C$, generic(C), $|F|$

$M \in K$ iff $M \in Ms$
  where $K \in$ {class _<_> extends _<_> { Ms _ }, new _<_> extends _<_>(_){Ms}}
$M \in C$ iff $M \in K$
  where K declared
        $K \in$ {class C<_> extends _<_> { _ _ }, new C<_> extends _<_>(_){_}}
generic(C) = C<dom(Δ)>
  where class C<Δ> extends _<_> { _ _ } declared
generic(C) = C<dom(Fs)>
  where new C<Fs> extends _<_>(_){_} declared
$|X$ funnel$| = X$ $|G| = G$

```
def K.methods : K -> List Method
  | .top dec => dec.methods
  | .loc _ _ _ _ _ ms => ms
def MethodIn (d : Decls) (mt : Method) (c : Name) :
   Prop :=
  exists k, Declared d k /\ k.name = c /\ List.Mem mt k.
     methods
def generic (d : Decls) (c : Name) : Option (List Name)
    :=
  match findK d c with
  | some (.top dec) => some (dom dec.bounds)
  | some (.loc _ fs _ _ _ _) => some (domF fs)
  | none => none
def Funnel.abs : Funnel -> Ty
  | .ground g => g
  | .funnel x => .var x
```

Mapping. $M \in K$ is `List.Mem M k.methods`; $M \in C$ is `MethodIn d M C`. `generic` returns the type variables Xs of C<Xs>. $|F|$ is `Funnel.abs`.

typeOf(v)

typeOf(v) = C<Gs>
  where $v \in$ {new C<Gs>(_), new C<Gs> extends _<_>(_){_}}

```
def typeOf : Expr -> Option Ty
  | .new c gs _ => some (.cls c gs)
  | .newLocal c fs _ _ _ _ => some (.cls c (fs.map
     Funnel.abs))
  | _ => none
```

Mapping. For a value of a local class, the ground funnels Gs become types through `Funnel.abs`.

A.4.4 Substitution and Renaming

Substitution of type variables

X' funnel$[Xs = Ts]$ = X' funnel if $X' \notin Xs$
X_i funnel$[X_1, \ldots, X_n = T_1, \ldots, T_n]$ = T_i
$G[Xs = Ts]$ = G
$X'[Xs = Ts]$ = X' if $X' \notin Xs$
$X_i[X_1, \ldots, X_n = T_1, \ldots, T_n]$ = T_i

The other cases go into every subterm, including the methods of a local class. In the domain of a substitution, bounds Δ stand for dom(Δ), and funnels Fs stand for their funnelled type variables; the ground funnels are ignored. Substitution of type variables into expressions happens only during reduction, where the substituted types are ground.

```
def mkSubst (xs : List Name) (ts : List Ty) : Option
   TSubst :=
  if xs.length = ts.length then some (xs.zip ts) else
     none
def mkSubstF (fs : List Funnel) (ts : List Ty) : Option
    TSubst :=
  if fs.length = ts.length then
    some ((fs.zip ts).filterMap fun
      | (.funnel x, t) => some (x, t)
      | (.ground _, _) => none)
  else none
def Ty.subst (sigma : TSubst) : Ty -> Ty
  | .var x => match sigma.lookup x with
    | some t => t
    | none => .var x
  | .cls c ts => .cls c (Ty.substList sigma ts)
def Funnel.subst (sigma : TSubst) : Funnel -> Funnel
  | .ground g => .ground (g.subst sigma)
  | .funnel x => match sigma.lookup x with
    | some t => .ground t
    | none => .funnel x
```

Mapping. $[\Delta = Ts]$ is `mkSubst (dom delta) ts`, and $[Fs = Ts]$ is `mkSubstF fs ts`; both are `none` when the lengths differ. `Expr.tsubst` and `Method.tsubst` go into every subterm. `Funnel.subst` does not check that the result is ground; `stepWellFormed` states it.

Substitution of variables

$x_i[x_1, \ldots, x_n = v_1, \ldots, v_n] = v_i$

(new C<Fs> extends C′<Ts>(es){Ms})[xs = vs]
= new C<Fs> extends C′<Ts>(es[xs = vs]){Ms[xs = vs]} if this ∉ xs

(new C<Fs> extends C′<Ts>(es){Ms})[xs, this, xs′ = vs, v, vs′]
= new C<Fs> extends C′<Ts>(es[xs, this, xs′ = vs, v, vs′]){Ms[xs, xs′ = vs, vs′]}

The other cases go into every subterm.

```
def Expr.vsubst (sigma : VSubst) : Expr -> Expr
  | .var x => match sigma.lookup x with
    | some v => v
    | none => .var x
  ...
  | .newLocal c fs c' ts es ms =>
    .newLocal c fs c' ts (Expr.vsubstList sigma es)
      (Method.vsubstList (sigma.filter fun p => p.1 !=
         thisName) ms)
def Method.vsubst (sigma : VSubst) : Method -> Method
  | .mk delta t m ps e => .mk delta t m ps (e.vsubst
     sigma)
```

Mapping. The two cases of the local class are one case: removing **this** from the substitution changes nothing when **this** is not in it.

fresh(e, e') and fresh(M, M')

The names bound in an expression e are the variable of every **if** occurring in e, and the names bound in every method of a local class occurring in e. The names bound in a method M are the type variables of its signature, its parameters, and the names bound in its body. fresh(e, e') holds when e' is e where every name bound in e is renamed into a fresh name, that is a name occurring nowhere in the program; distinct binders get distinct fresh names. fresh(M, M') holds when M' is M renamed in the same way. The variable **this** is never renamed.

```
def freshNames (avoid ns : List Name) : Bool :=
  decide ns.Nodup && ns.all fun n => !(avoid.contains n
     )
def Fresh (avoid : List Name) (e e' : Expr) : Prop :=
  exists ns, Expr.freshen [] [] ns e = some (e', []) /\
    freshNames avoid ns = true
def FreshM (avoid : List Name) (mt mt' : Method) : Prop
    :=
  exists ns, Method.freshen [] [] ns mt = some (mt',
     []) /\
    freshNames avoid ns = true
def avoidNames (d : Decls) (e : Expr) : List Name := d.
   names ++ e.names
```

Mapping. `Expr.freshen rv rt ns e` walks e, takes the next name of `ns` at every binder, and renames the bound occurrences: `rv` renames variables, `rt` type variables. The rules pass `avoidNames d redex`: the names of Decls and of the redex, fewer than all the names of the program. This is enough for A7, since the hole of an evaluation context is under no binder.

A.4.5 Reduction

simpleFieldFound

$$\frac{\texttt{class } C\texttt{<_>} \texttt{ extends } _\texttt{<_>} \texttt{ \{ } _\ T_1\ f_1\texttt{;} \ldots T_n\ f_n\texttt{; \}} \text{ declared} \quad f_i \in f_1, \ldots, f_n}{\texttt{new } C\texttt{<}Gs\texttt{>}(v_1, \ldots, v_n, vs)\texttt{.}f_i \to v_i}$$

```
| simpleFieldFound {c gs vs1 vs f v} {dec : Dec} :
    List.Mem dec d -> dec.name = c ->
    vs1.length = dec.fields.length ->
    (Expr.new c gs (vs1 ++ vs)).isValue = true ->
    ((fieldNames dec).zip vs1).lookup f = some v ->
    Step d (.field (.new c gs (vs1 ++ vs)) f) v
```

Mapping. $v_1, \ldots, v_n$ is `vs1`, as long as the fields; f_i and v_i are found by `lookup` in the pairs of field names and values, which A2 makes unique.

simpleFieldSuper

$$\frac{\texttt{class } C\texttt{<}\Delta\texttt{>} \texttt{ extends } C'\texttt{<}Ts\texttt{>} \texttt{ \{ } _\ T_1\ f_1\texttt{;} \ldots T_n\ f_n\texttt{; \}} \text{ declared} \quad f \notin f_1, \ldots, f_n \qquad C \neq \texttt{Object}}{\texttt{new } C\texttt{<}Gs\texttt{>}(v_1, \ldots, v_n, vs)\texttt{.}f \to \texttt{new } C'\texttt{<}Ts[\Delta = Gs]\texttt{>}(vs)\texttt{.}f}$$

```
| simpleFieldSuper {c gs vs1 vs f sigma} {dec : Dec} :
    List.Mem dec d -> dec.name = c ->
    vs1.length = dec.fields.length ->
    (Expr.new c gs (vs1 ++ vs)).isValue = true ->
    (fieldNames dec).contains f = false ->
    Not (c = objectName) ->
    mkSubst (dom dec.bounds) gs = some sigma ->
    Step d (.field (.new c gs (vs1 ++ vs)) f)
      (.field (.new dec.sup (Ty.substList sigma dec.
         supArgs) vs) f)
```

Mapping. $[\Delta = Gs]$ is `sigma`.

innerFieldSuper

$$\texttt{new } C\texttt{<}Gs\texttt{>} \texttt{ extends } C'\texttt{<}Gs'\texttt{>}(vs)\{Ms\}.f \to \texttt{new } C'\texttt{<}Gs'\texttt{>}(vs).f$$

```
| innerFieldSuper {c fs c' gs vs ms f} :
    (Expr.newLocal c fs c' gs vs ms).isValue = true ->
    Step d (.field (.newLocal c fs c' gs vs ms) f) (.
       field (.new c' gs vs) f)
```

Mapping. The premise `isValue` is the grammar of v and G.

ifThen

$$\frac{\emptyset \vdash \mathsf{typeOf}(v) <: C'\texttt{<_>} \qquad \mathsf{fresh}(e_1, e_1')}{\texttt{if } (v \texttt{ instanceof } C'\texttt{<}Ts\texttt{>}\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \to e_1'[x = v]}$$

```
| ifThen {v c' ts ts' tv x e1 e1' e2} :
    v.isValue = true -> typeOf v = some tv ->
    Sub d [] tv (.cls c' ts') ->
    Fresh (avoidNames d (.ifInst v (.cls c' ts) x e1 e2
       )) e1 e1' ->
    Step d (.ifInst v (.cls c' ts) x e1 e2) (e1'.vsubst
        [(x, v)])
```

Mapping. The `_` in C'`<_>` is the implicit `ts'`.

ifElse

$$\frac{\emptyset \not\vdash \mathsf{typeOf}(v) <: C'\texttt{<_>}}{\texttt{if } (v \texttt{ instanceof } C'\texttt{<}Ts\texttt{>}\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} \to e_2}$$

```
| ifElse {v c' ts tv x e1 e2} :
    v.isValue = true -> typeOf v = some tv ->
    (forall ts', Not (Sub d [] tv (.cls c' ts'))) ->
    Step d (.ifInst v (.cls c' ts) x e1 e2) e2
```

Mapping. $\not\vdash \ldots <: C'$`<_>`: no type arguments make the subtyping hold.

simpleAddSuper

$$\frac{v = \texttt{new } C\texttt{<}Gs\texttt{>(_)}}{v\texttt{.<}Gs'\texttt{>}m(vs) \to v\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(vs)}$$

```
| simpleAddSuper {c gs vs0 gs' m vs} :
    (Expr.new c gs vs0).isValue = true -> Ty.allGround
       gs' = true ->
    Expr.allValues vs = true ->
    Step d (.call (.new c gs vs0) gs' m vs)
      (.superCall (.new c gs vs0) (.cls c gs) gs' m vs)
```

Mapping. The premise $v =$ `new` C`<`Gs`>(_)` is the shape of the receiver in the conclusion.

simpleFound

$$\frac{\begin{array}{c}\texttt{class } C\texttt{<}\Delta\texttt{>} \texttt{ extends } \texttt{_<_>}\ \{\ Ms\ _\ \}\ \mathsf{declared} \\ M \in Ms \\ \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return } e_0\texttt{;}\ \}) \\ e = e_0[\Delta, \Delta' = Gs, Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]\end{array}}{v_0\texttt{.[}C\texttt{<}Gs\texttt{>]<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

```
| simpleFound {v0 c gs gs' m vs mt delta' t0 ps e0
   sigma} {dec : Dec} :
    List.Mem dec d -> dec.name = c ->
    v0.isValue = true -> Ty.allGround gs = true -> Ty.
       allGround gs' = true ->
    Expr.allValues vs = true ->
    List.Mem mt dec.methods ->
    FreshM (avoidNames d (.superCall v0 (.cls c gs) gs'
        m vs)) mt
      (.mk delta' t0 m ps e0) ->
    mkSubst (dom dec.bounds ++ dom delta') (gs ++ gs')
       = some sigma ->
    ps.length = vs.length ->
    Step d (.superCall v0 (.cls c gs) gs' m vs)
      ((e0.tsubst sigma).vsubst ((thisName, v0) :: (ps.
         map Prod.snd).zip vs))
```

Mapping. $[\Delta, \Delta' = Gs, Gs']$ is `sigma`; $x_1, \ldots, x_n$ is `ps.map Prod.snd`, as many as the arguments `vs`.

innerFound

$$\frac{v_0 = \texttt{new}\ _\texttt{<}_\texttt{>}\ \texttt{extends}\ _\texttt{<}_\texttt{>}(_)\{Ms\} \quad M \in Ms \quad \mathsf{fresh}(M, \texttt{<}\Delta'\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e_0\texttt{;}\ \}) \quad e = e_0[\Delta' = Gs'][\texttt{this}, x_1, \ldots, x_n = v_0, \ldots, v_n]}{v_0.\texttt{<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to e}$$

```
| innerFound {c fs c' gs vs0 ms gs' m vs mt delta' t0
   ps e0 sigma} :
    (Expr.newLocal c fs c' gs vs0 ms).isValue = true ->
        Ty.allGround gs' = true ->
    Expr.allValues vs = true ->
    List.Mem mt ms ->
    FreshM (avoidNames d (.call (.newLocal c fs c' gs
       vs0 ms) gs' m vs)) mt
      (.mk delta' t0 m ps e0) ->
    mkSubst (dom delta') gs' = some sigma ->
    ps.length = vs.length ->
    Step d (.call (.newLocal c fs c' gs vs0 ms) gs' m
       vs)
      ((e0.tsubst sigma).vsubst
        ((thisName, .newLocal c fs c' gs vs0 ms) :: (ps.
           map Prod.snd).zip vs))
```

Mapping. As SIMPLEFOUND, with v_0 written out as `.newLocal c fs c' gs vs0 ms`.

innerSuper

$$\frac{v_0 = \texttt{new}\ _\texttt{<}_\texttt{>}\ \texttt{extends}\ C\texttt{<}Gs\texttt{>}(_)\{Ms\} \quad m \notin Ms}{v_0.\texttt{<}Gs'\texttt{>}m(v_1, \ldots, v_n) \to v_0.[C\texttt{<}Gs\texttt{>}]\texttt{<}Gs'\texttt{>}m(v_1, \ldots, v_n)}$$

```
| innerSuper {c fs c' gs vs0 ms gs' m vs} :
    (Expr.newLocal c fs c' gs vs0 ms).isValue = true ->
        Ty.allGround gs' = true ->
    Expr.allValues vs = true ->
    (ms.map Method.name).contains m = false ->
    Step d (.call (.newLocal c fs c' gs vs0 ms) gs' m
       vs)
      (.superCall (.newLocal c fs c' gs vs0 ms) (.cls c'
         gs) gs' m vs)
```

Mapping. The superclass C of the paper is `c'` in Lean.

searchSuper

$$\frac{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ _\ \}\ \text{declared} \quad m \notin Ms \qquad C \neq \texttt{Object}}{v.[C\texttt{<}Gs\texttt{>}]\texttt{<}Gs'\texttt{>}m(vs) \to v.[C'\texttt{<}Ts[\Delta = Gs]\texttt{>}]\texttt{<}Gs'\texttt{>}m(vs)}$$

```
| searchSuper {v c gs gs' m vs sigma} {dec : Dec} :
    List.Mem dec d -> dec.name = c ->
    v.isValue = true -> Ty.allGround gs = true -> Ty.
       allGround gs' = true ->
    Expr.allValues vs = true ->
    (dec.methods.map Method.name).contains m = false ->
    Not (c = objectName) ->
    mkSubst (dom dec.bounds) gs = some sigma ->
    Step d (.superCall v (.cls c gs) gs' m vs)
      (.superCall v (.cls dec.sup (Ty.substList sigma
         dec.supArgs)) gs' m vs)
```

Mapping. $m \notin Ms$ is `contains m = false` on the method names.

ctx and $\to^*$

$$\frac{e \to e'}{\mathcal{E}[e] \to \mathcal{E}[e']}$$

$e_0 \to^* e$: zero or more steps.

```
| ctx {E : ECtx} {e e'} :
    E.ok = true -> Step d e e' -> Step d (E.plug e) (E.
       plug e')
inductive Steps (d : Decls) : Expr -> Expr -> Prop
  | refl {e} : Steps d e e
  | step {e e' e''} : Step d e e' -> Steps d e' e'' ->
     Steps d e e''
```

Mapping. $\mathcal{E}$ is `E` with `E.ok`; $\to^*$ is `Steps d`.

A.4.6 Subtyping and Kinding

sub-refl, sub-var

$$\frac{}{\Delta \vdash T <: T} \qquad \frac{X \texttt{ extends } T \in \Delta \qquad \Delta \vdash T <: T'}{\Delta \vdash X <: T'}$$

```
inductive Sub (d : Decls) (delta : Bounds) : Ty -> Ty
   -> Prop
  | refl {t} : Sub d delta t t
  | var {x t t'} :
     delta.lookup x = some t -> Sub d delta t t' ->
        Sub d delta (.var x) t'
```

Mapping. X `extends` $T \in \Delta$ is `delta.lookup x = some t`; by A5 there is one bound per X.

sub-inherit

$$\frac{\texttt{class } C\texttt{<}\Delta_0\texttt{>} \texttt{ extends } C'\texttt{<}Ts\texttt{>} \texttt{ \{ _ _ \}} \text{ declared} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}[\Delta_0 = Ts_0] <: T'}{\Delta \vdash C\texttt{<}Ts_0\texttt{>} <: T'}$$

```
  | inherit {c ts0 t' sigma} {dec : Dec} :
     List.Mem dec d -> dec.name = c ->
     mkSubst (dom dec.bounds) ts0 = some sigma ->
     Sub d delta ((Ty.cls dec.sup dec.supArgs).subst
        sigma) t' ->
     Sub d delta (.cls c ts0) t'
```

Mapping. Δ_0 is `dec.bounds`; $C'\texttt{<}Ts\texttt{>}$ is `Ty.cls dec.sup dec.supArgs`.

sub-inherit-local

$$\frac{\texttt{new } C\texttt{<}Fs\texttt{>} \texttt{ extends } C'\texttt{<}Ts'\texttt{>(_)\{_\}} \text{ declared} \qquad \Delta \vdash C'\texttt{<}Ts'\texttt{>}[Fs = Ts] <: T'}{\Delta \vdash C\texttt{<}Ts\texttt{>} <: T'}$$

```
  | inheritLocal {c fs c' ts' es ms ts t' sigma} :
     Declared d (.loc c fs c' ts' es ms) ->
     mkSubstF fs ts = some sigma ->
     Sub d delta ((Ty.cls c' ts').subst sigma) t' ->
     Sub d delta (.cls c ts) t'
```

Mapping. $[Fs = Ts]$ is `mkSubstF fs ts`.

wf-var, wf-class

$$\frac{X \texttt{ extends } _ \in \Delta}{\Delta \vdash X \text{ Ok}} \qquad \frac{\text{scope}(C\texttt{<}Ts\texttt{>}) = \Delta_0; Ts_0 \qquad \Delta \vdash Ts \text{ Ok} \qquad \Delta \vdash Ts_0 <: \text{cod}(\Delta_0)[\Delta_0 = Ts_0]}{\Delta \vdash C\texttt{<}Ts\texttt{>} \text{ Ok}}$$

```
inductive Kind (d : Decls) (delta : Bounds) : Ty ->
   Prop
  | var {x} : (delta.lookup x).isSome = true -> Kind d
     delta (.var x)
  | cls {c ts delta0 ts0 sigma} :
     scope d c ts = some (delta0, ts0) ->
     (forall t, List.Mem t ts -> Kind d delta t) ->
     mkSubst (dom delta0) ts0 = some sigma ->
     (forall p, List.Mem p (ts0.zip (Ty.substList
        sigma (cod delta0))) ->
       Sub d delta p.1 p.2) ->
     Kind d delta (.cls c ts)
```

Mapping. The subtyping on sequences is one `Sub` for each pair of `zip`.

A.4.7 Checked Cast

cast-up, cast-var

$$\frac{\Delta \vdash T <: T'}{\Delta \vdash T \rightsquigarrow T'} \qquad \frac{X \texttt{ extends } T \in \Delta \qquad \Delta \vdash T \rightsquigarrow T'}{\Delta \vdash X \rightsquigarrow T'}$$

```
inductive Cast (d : Decls) (delta : Bounds) : Ty -> Ty
   -> Prop
  | up {t t'} : Sub d delta t t' -> Cast d delta t t'
  | var {x t t'} :
     delta.lookup x = some t -> Cast d delta t t' ->
        Cast d delta (.var x) t'
```

Mapping. $\Delta \vdash T \rightsquigarrow T'$ is `Cast d delta T T'`.

cast-down

$$\frac{\Delta \vdash C\texttt{<}Ts\texttt{>} <: C'\texttt{<}Ts_0\texttt{>} \qquad \text{generic}(C) = C\texttt{<}Xs\texttt{>} \qquad \emptyset \vdash C\texttt{<}Xs\texttt{>} <: C'\texttt{<}Ts'\texttt{>} \qquad Xs \subseteq \mathsf{FTV}(Ts')}{\Delta \vdash C'\texttt{<}Ts_0\texttt{>} \rightsquigarrow C\texttt{<}Ts\texttt{>}}$$

```
| down {c ts c' ts0 xs ts'} :
    Sub d delta (.cls c ts) (.cls c' ts0) ->
    generic d c = some xs ->
    Sub d [] (.cls c (vars xs)) (.cls c' ts') ->
    (forall x, List.Mem x xs -> List.Mem x (Ty.
       ftvList ts')) ->
    Cast d delta (.cls c' ts0) (.cls c ts)
```

Mapping. Xs is `xs`; C`<`Xs`>` is `.cls c (vars xs)`.

A.4.8 Typing

subs, var, new-object

$$\frac{\Delta;\Gamma \vdash e : T_1 \qquad \Delta \vdash T_2 \text{ Ok} \qquad \Delta \vdash T_1 <: T_2}{\Delta;\Gamma \vdash e : T_2}$$

$$\frac{}{\Delta;\Gamma \vdash x : \Gamma(x)}$$

$$\frac{}{\Delta;\Gamma \vdash \texttt{new Object<>()} : \texttt{Object<>}}$$

```
inductive Typing (d : Decls) (java : Bool) : Bounds ->
   Ctx -> Expr -> Ty -> Prop
 | subs {delta gamma e t1 t2} :
    Typing d java delta gamma e t1 -> Kind d delta t2
       -> Sub d delta t1 t2 ->
    Typing d java delta gamma e t2
 | var {delta gamma x t} :
    gamma.lookup x = some t -> Typing d java delta
       gamma (.var x) t
 | newObject {delta gamma} :
    Typing d java delta gamma (.new objectName [] [])
       (.cls objectName [])
```

Mapping. The flag `java` selects the variant: `false` is ζ(funnelled), `true` is ζ(Java). Only NEW-LOCAL and NEW-LOCAL-JAVA read it.

declField

$$\frac{\Delta;\Gamma \vdash e : C\texttt{<}Ts'\texttt{>} \qquad \texttt{class } C\texttt{<}\Delta'\texttt{> extends _<_> \{ _ } Ss \texttt{ \}} \text{ declared} \qquad T\ f; \in Ss}{\Delta;\Gamma \vdash e.f : T[\Delta' = Ts']}$$

```
| declField {delta gamma e c ts' t f sigma} {dec :
   Dec} :
    Typing d java delta gamma e (.cls c ts') ->
    List.Mem dec d -> dec.name = c ->
    List.Mem (t, f) dec.fields ->
    mkSubst (dom dec.bounds) ts' = some sigma ->
    Typing d java delta gamma (.field e f) (t.subst
       sigma)
```

Mapping. Δ' is `dec.bounds`; Ss is `dec.fields`.

call

$$\frac{\begin{array}{c}\Delta;\Gamma \vdash e_0 : C_0\texttt{<}Ts_0\texttt{>} \\ \text{scope}(C_0\texttt{<}Ts_0\texttt{>}) = \Delta_0; Ts_2 \\ \texttt{<}\Delta_1\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \texttt{\{ _ \}} \in C_0 \\ \forall i \in 0..n.\ T_i' = T_i[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta \vdash Ts_1 \text{ Ok} \\ \Delta \vdash Ts_1 <: \text{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_2, Ts_1] \\ \Delta;\Gamma \vdash e_1, \ldots, e_n : T_1', \ldots, T_n'\end{array}}{\Delta;\Gamma \vdash e_0.\texttt{<}Ts_1\texttt{>}m(e_1, \ldots, e_n) : T_0'}$$

```
| call {delta gamma e0 c0 ts0 delta0 ts2 delta1 t0 m
   ps body ts1 es sigma} :
    Typing d java delta gamma e0 (.cls c0 ts0) ->
    scope d c0 ts0 = some (delta0, ts2) ->
    MethodIn d (.mk delta1 t0 m ps body) c0 ->
    mkSubst (dom delta0 ++ dom delta1) (ts2 ++ ts1) =
       some sigma ->
    (forall t, List.Mem t ts1 -> Kind d delta t) ->
    (forall p, List.Mem p (ts1.zip (Ty.substList
       sigma (cod delta1))) ->
      Sub d delta p.1 p.2) ->
    es.length = ps.length ->
    (forall p, List.Mem p (es.zip (Ty.substList sigma
       (ps.map Prod.fst))) ->
      Typing d java delta gamma p.1 p.2) ->
    Typing d java delta gamma (.call e0 ts1 m es) (t0.
       subst sigma)
```

Mapping. $[\Delta_0, \Delta_1 = Ts_2, Ts_1]$ is `sigma`; T_i' is written out as `subst sigma`.

call-lb-runtime

$$\frac{\begin{array}{c}\Delta \vdash C\texttt{<}Ts\texttt{>} <: C_0\texttt{<}Ts_0\texttt{>} \\ \texttt{class } C_0\texttt{<}\Delta_0\texttt{>} \texttt{ extends } _\texttt{<}_\texttt{>} \texttt{ \{ } _\ _ \texttt{ \} declared} \\ \texttt{<}\Delta_1\texttt{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \texttt{\{ } _ \texttt{ \}} \in C_0 \\ \forall i \in 0..n.\ T_i' = T_i[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \texttt{class } C\texttt{<}_\texttt{> extends } _\texttt{<}_\texttt{> \{ } _\ _ \texttt{ \} declared} \\ \Delta \vdash C\texttt{<}Ts\texttt{>}\ \mathrm{Ok} \qquad \Delta \vdash Ts_1\ \mathrm{Ok} \\ \Delta \vdash Ts_1 <: \mathrm{cod}(\Delta_1)[\Delta_0, \Delta_1 = Ts_0, Ts_1] \\ \Delta; \Gamma \vdash e_0, \ldots, e_n : C\texttt{<}Ts\texttt{>}, T_1', \ldots, T_n' \end{array}}{\Delta; \Gamma \vdash e_0\texttt{.[}C\texttt{<}Ts\texttt{>]<}Ts_1\texttt{>}m(e_1, \ldots, e_n) : T_0'}$$

```
| superCall {delta gamma e0 c ts c0 ts0 delta1 t0 m
   ps body ts1 es sigma}
    {dec0 dec : Dec} :
    Sub d delta (.cls c ts) (.cls c0 ts0) ->
    List.Mem dec0 d -> dec0.name = c0 ->
    MethodIn d (.mk delta1 t0 m ps body) c0 ->
    mkSubst (dom dec0.bounds ++ dom delta1) (ts0 ++
       ts1) = some sigma ->
    List.Mem dec d -> dec.name = c ->
    Kind d delta (.cls c ts) ->
    (forall t, List.Mem t ts1 -> Kind d delta t) ->
    (forall p, List.Mem p (ts1.zip (Ty.substList
       sigma (cod delta1))) ->
      Sub d delta p.1 p.2) ->
    Typing d java delta gamma e0 (.cls c ts) ->
    es.length = ps.length ->
    (forall p, List.Mem p (es.zip (Ty.substList sigma
        (ps.map Prod.fst))) ->
      Typing d java delta gamma p.1 p.2) ->
    Typing d java delta gamma (.superCall e0 (.cls c
       ts) ts1 m es) (t0.subst sigma)
```

Mapping. Δ_0 is `dec0.bounds`; the premise on e_0 is the first element of the paper's sequence.

new-top

$$\frac{\begin{array}{c}\texttt{class } C\texttt{<}\Delta'\texttt{> extends } C'\texttt{<}Ts\texttt{> \{ } _\ T_1\ f_1\texttt{; } \ldots\ T_n\ f_n\texttt{; \} declared} \\ \Delta \vdash C\texttt{<}Ts'\texttt{>}\ \mathrm{Ok} \\ \Delta; \Gamma \vdash e_1, \ldots, e_n : T_1, \ldots, T_n[\Delta' = Ts'] \\ \Delta; \Gamma \vdash \texttt{new } C'\texttt{<}Ts[\Delta' = Ts']\texttt{>(}es\texttt{)} : _ \end{array}}{\Delta; \Gamma \vdash \texttt{new } C\texttt{<}Ts'\texttt{>(}e_1, \ldots, e_n, es\texttt{)} : C\texttt{<}Ts'\texttt{>}}$$

```
| newTop {delta gamma c ts' es1 es2 sigma t''} {dec :
    Dec} :
    List.Mem dec d -> dec.name = c ->
    Kind d delta (.cls c ts') ->
    mkSubst (dom dec.bounds) ts' = some sigma ->
    es1.length = dec.fields.length ->
    (forall p, List.Mem p (es1.zip (Ty.substList
       sigma (dec.fields.map Prod.fst))) ->
      Typing d java delta gamma p.1 p.2) ->
    Typing d java delta gamma (.new dec.sup (Ty.
       substList sigma dec.supArgs) es2) t'' ->
    Typing d java delta gamma (.new c ts' (es1 ++ es2
       )) (.cls c ts')
```

Mapping. $e_1, \ldots, e_n$ is `es1`, as long as the fields; es is `es2`; the _ type is `t"`.

new-local

$$\frac{\begin{array}{c}\texttt{new } C\texttt{<}Fs\texttt{> extends } C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}}\ \mathrm{consistent} \\ Fs \overset{\Delta}{\Longleftrightarrow} |Fs| = \Delta'; _ \\ \mathsf{FTV}(\mathrm{cod}(\Delta')) \subseteq \mathrm{dom}(\Delta') \\ \Delta' \vdash C\texttt{<}|Fs|\texttt{>}\ \mathrm{Ok} \\ \Delta' \vdash C_0\texttt{<}Ts\texttt{>}\ \mathrm{Ok} \qquad \Delta; \Gamma \vdash \texttt{new } C_0\texttt{<}Ts\texttt{>(}es\texttt{)} : _ \\ \Delta'; \Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}, C\texttt{<}|Fs|\texttt{> this} \vdash Ms\ \mathrm{Ok} \\ \mathsf{overrideOk}(\Delta', C_0\texttt{<}Ts\texttt{>}, Ms) \end{array}}{\Delta; \Gamma \vdash \texttt{new } C\texttt{<}Fs\texttt{> extends } C_0\texttt{<}Ts\texttt{>(}es\texttt{)\{}Ms\texttt{\}} : C\texttt{<}|Fs|\texttt{>}}$$

For a set of type variables Xs, $\Gamma|_{Xs}$ keeps the variables $T\ x$ of Γ such that $\mathsf{FTV}(T) \subseteq Xs$. $\Gamma \setminus x$ is Γ without the variable x.

```
| newLocal {delta gamma c fs c0 ts es ms delta' ts''
   t''} :
    java = false ->
    Consistent d (.loc c fs c0 ts es ms) ->
    pairing delta fs (fs.map Funnel.abs) = some (
       delta', ts'') ->
    (forall x, List.Mem x (Ty.ftvList (cod delta')) ->
        List.Mem x (dom delta')) ->
    Kind d delta' (.cls c (fs.map Funnel.abs)) ->
    Kind d delta' (.cls c0 ts) ->
    Typing d java delta gamma (.new c0 ts es) t'' ->
    (forall mt, List.Mem mt ms ->
      MethodOk d java delta'
        (restrict gamma (Ty.ftvList (fs.map Funnel.abs
           )) ++
          [(thisName, .cls c (fs.map Funnel.abs))]) mt
            ) ->
    OverrideOk d delta' (.cls c0 ts) ms ->
    Typing d java delta gamma (.newLocal c fs c0 ts
       es ms) (.cls c (fs.map Funnel.abs))
def restrict (gamma : Ctx) (xs : List Name) : Ctx :=
  gamma.filter fun p => p.1 != thisName && p.2.ftv.all
     fun x => xs.contains x
```

Mapping. $|Fs|$ is `fs.map Funnel.abs`; $\Gamma|_{\mathsf{FTV}(|Fs|)} \setminus \texttt{this}$ is `restrict`; adding $C\texttt{<}|Fs|\texttt{> this}$ appends a pair, and `lookup` finds it since `restrict` removed the old **`this`**.

new-local-java

$$\frac{\Delta \vdash C'\texttt{<}Ts\texttt{>}\ \text{Ok} \qquad \Delta;\Gamma \vdash \texttt{new}\ C'\texttt{<}Ts\texttt{>}(es) : _ \qquad \Delta;\Gamma \setminus \texttt{this}, C\texttt{<>}\ \texttt{this} \vdash Ms\ \text{Ok} \qquad \text{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\Delta;\Gamma \vdash \texttt{new}\ C\texttt{<>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}(es)\{Ms\} : C\texttt{<>}}$$

```
  | newLocalJava {delta gamma c c0 ts es ms t''} :
      java = true ->
      Kind d delta (.cls c0 ts) ->
      Typing d java delta gamma (.new c0 ts es) t'' ->
      (forall mt, List.Mem mt ms ->
        MethodOk d java delta (withoutThis gamma ++ [(
            thisName, .cls c [])]) mt) ->
      OverrideOk d delta (.cls c0 ts) ms ->
      Typing d java delta gamma (.newLocal c [] c0 ts
        es ms) (.cls c [])
def withoutThis (gamma : Ctx) : Ctx := gamma.filter fun
    p => p.1 != thisName
```

Mapping. C' of the paper is `c0` in Lean.

if-instanceof

$$\frac{\Delta;\Gamma \vdash e_0 : T_0 \qquad \Delta \vdash T\ \text{Ok} \qquad \Delta \vdash T_0 \rightsquigarrow T \qquad \Delta;\Gamma, T\ x \vdash e_1 : T' \qquad \Delta;\Gamma \vdash e_2 : T'}{\Delta;\Gamma \vdash \texttt{if}\ (e_0\ \texttt{instanceof}\ T\ x)\ \{\ e_1\ \}\ \texttt{else}\ \{\ e_2\ \} : T'}$$

```
  | ifInst {delta gamma e0 t0 t x e1 e2 t'} :
      Typing d java delta gamma e0 t0 -> Kind d delta t
        -> Cast d delta t0 t ->
      Typing d java delta (gamma ++ [(x, t)]) e1 t' ->
      Typing d java delta gamma e2 t' ->
      Typing d java delta gamma (.ifInst e0 t x e1 e2)
        t'
```

Mapping. $\Gamma, T\ x$ is `gamma ++ [(x, t)]`; by A7, x is not already in Γ.

A.4.9 Consistency and Overriding

K consistent

$\texttt{new}\ C\texttt{<}Fs\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts\texttt{>}(es)\{Ms\}$ consistent
where $\texttt{new}\ C\texttt{<}Fs'\texttt{>}\ \texttt{extends}\ C_0\texttt{<}Ts'\texttt{>}(_)\{Ms'\}$ declared
$Ts = Ts'[Fs' = |Fs|]$
$\text{dom}(Ms') = \text{dom}(Ms)$
$\text{overrideOk}(C\texttt{<}|Fs|\texttt{>}, Ms)$

$\text{dom}(Ms)$ is the set of the names of the methods in Ms.

```
def Consistent (d : Decls) : K -> Prop
  | .loc c fs c0 ts _ ms =>
    exists fs' ts' es' ms' sigma,
      Declared d (.loc c fs' c0 ts' es' ms') /\
      mkSubstF fs' (fs.map Funnel.abs) = some sigma /\
      ts = Ty.substList sigma ts' /\
      (forall m, List.Mem m (ms'.map Method.name) <->
        List.Mem m (ms.map Method.name)) /\
      forall mt, List.Mem mt ms -> OverrideOk1 d (.cls
        c (fs.map Funnel.abs)) mt
  | .top _ => False
```

Mapping. $\text{overrideOk}(C\texttt{<}|Fs|\texttt{>}, Ms)$ on the sequence Ms is one `OverrideOk1` per method.

$\text{overrideOk}(\Delta, T, Ms)$

$\text{overrideOk}(\Delta, T, Ms)$ iff $\text{overrideOk}(C\texttt{<}Ts\texttt{>}, M)$
for every $M \in Ms$ and every $C\texttt{<}Ts\texttt{>}$ such that $\Delta \vdash T <: C\texttt{<}Ts\texttt{>}$

```
def OverrideOk (d : Decls) (delta : Bounds) (t : Ty) (
   ms : List Method) : Prop :=
  forall mt c ts, List.Mem mt ms -> Sub d delta t (.cls
      c ts) ->
    OverrideOk1 d (.cls c ts) mt
```

Mapping. $\text{overrideOk}(T, M)$ is `OverrideOk1`.

overrideOk(T, M)

overrideOk(C<Ts>, <_>_ m(_) { _ }) iff $m \notin Ms$
  where class C<_> extends _<_> { Ms _ } declared

overrideOk(C<Ts>, <Δ'>T_0' m(T_1' x_1, …, T_n' x_n) { _ })
  where class C<Δ> extends _<_> { Ms _ } declared
  <Δ''>T_0 m(T_1 x_1', …, T_n x_n') { _ } $\in Ms$
  $\mathrm{cod}(\Delta'')[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
  $T_0, \ldots, T_n[\Delta, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0', \ldots, T_n'$

overrideOk(C<Ts>, <Δ'>T_0' m(T_1' x_1, …, T_n' x_n) { _ })
  where new C<Fs> extends _<_>(_){Ms} declared
  <Δ''>T_0 m(T_1 x_1', …, T_n x_n') { _ } $\in Ms$
  $\mathrm{cod}(\Delta'')[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = \mathrm{cod}(\Delta')$
  $T_0, \ldots, T_n[Fs, \Delta'' = Ts, \mathrm{dom}(\Delta')] = T_0', \ldots, T_n'$

```
def OverrideOk1 (d : Decls) (t : Ty) (mt : Method) :
   Prop :=
  match t, mt with
  | .cls c ts, .mk delta' t0' m ps' _ =>
    (exists dec : Dec, List.Mem dec d /\ dec.name = c
       /\
      Not (List.Mem m (dec.methods.map Method.name)))
    \/ (exists (dec : Dec) (delta'' : Bounds) (t0 : Ty)
          (ps : List (Prod Ty Name)) (e : Expr) (sigma :
              TSubst),
          List.Mem dec d /\ dec.name = c /\
          List.Mem (Method.mk delta'' t0 m ps e) dec.
             methods /\
          mkSubst (dom dec.bounds ++ dom delta'') (ts ++
              vars (dom delta''))
            = some sigma /\
          Ty.substList sigma (cod delta'') = cod delta'
             /\
          Ty.substList sigma (t0 :: ps.map Prod.fst) =
             t0' :: ps'.map Prod.fst)
    \/ (exists fs c0 ts0 es ms delta'' t0 ps e sigmaF
       sigma,
          Declared d (.loc c fs c0 ts0 es ms) /\
          List.Mem (Method.mk delta'' t0 m ps e) ms /\
          mkSubstF fs ts = some sigmaF /\
          mkSubst (dom delta'') (vars (dom delta'')) =
             some sigma /\
          Ty.substList (sigmaF ++ sigma) (cod delta'') =
              cod delta' /\
          Ty.substList (sigmaF ++ sigma) (t0 :: ps.map
             Prod.fst)
            = t0' :: ps'.map Prod.fst)
  | .var _, _ => False
```

Mapping. The three cases are the three disjuncts. Comparing the lists $T_0, \ldots, T_n$ and $T_0', \ldots, T_n'$ also requires the same number of parameters, as the paper does by writing n in both methods. $[Fs, \Delta'' = \ldots]$ is `sigmaF ++ sigma`, whose domains are disjoint by A7.

A.4.10 Methods, Classes and Programs

method-ok

$$\frac{\Delta, \Delta' \vdash \mathrm{cod}(\Delta')\ \mathrm{Ok} \qquad \Delta, \Delta' \vdash T_0, \ldots, T_n\ \mathrm{Ok} \qquad \Delta, \Delta'; \Gamma, T_1\ x_1, \ldots, T_n\ x_n \vdash e : T_0}{\Delta; \Gamma \vdash \text{<}\Delta'\text{>}T_0\ m(T_1\ x_1, \ldots, T_n\ x_n)\ \{\ \texttt{return}\ e;\ \}\ \mathrm{Ok}}$$

```
inductive MethodOk (d : Decls) (java : Bool) : Bounds
   -> Ctx -> Method -> Prop
  | mk {delta gamma delta' t0 m ps e} :
     (forall t, List.Mem t (cod delta') -> Kind d (
        delta ++ delta') t) ->
     (forall t, List.Mem t (t0 :: ps.map Prod.fst) ->
        Kind d (delta ++ delta') t) ->
     Typing d java (delta ++ delta') (gamma ++ ps.map
        fun p => (p.2, p.1)) e t0 ->
     MethodOk d java delta gamma (.mk delta' t0 m ps e
        )
```

Mapping. `MethodOk` is defined together with `Typing`, since NEW-LOCAL has a premise $\Delta'; \ldots \vdash Ms$ Ok.

class-ok

$$\frac{\Delta \vdash \mathrm{cod}(\Delta)\ \mathrm{Ok} \qquad \Delta \vdash C'\texttt{<}Ts\texttt{>}, T_1, \ldots, T_n\ \mathrm{Ok} \qquad \Delta; C\texttt{<}\mathrm{dom}(\Delta)\texttt{>}\ \texttt{this} \vdash Ms\ \mathrm{Ok} \qquad \mathsf{overrideOk}(\Delta, C'\texttt{<}Ts\texttt{>}, Ms)}{\texttt{class}\ C\texttt{<}\Delta\texttt{>}\ \texttt{extends}\ C'\texttt{<}Ts\texttt{>}\ \{\ Ms\ T_1\ f_1; \ldots T_n\ f_n;\ \}\ \mathrm{Ok}}$$

```
def ClassOk (d : Decls) (java : Bool) (dec : Dec) :
   Prop :=
  (forall t, List.Mem t (cod dec.bounds) -> Kind d dec.
     bounds t) /\
  (forall t, List.Mem t (Ty.cls dec.sup dec.supArgs ::
     dec.fields.map Prod.fst) ->
    Kind d dec.bounds t) /\
  (forall mt, List.Mem mt dec.methods ->
    MethodOk d java dec.bounds [(thisName, .cls dec.
       name (vars (dom dec.bounds)))] mt) /\
  OverrideOk d dec.bounds (.cls dec.sup dec.supArgs)
     dec.methods
```

Mapping. A rule with one conclusion and no recursion is a definition in Lean: the four premises are the four conjuncts.

prog-ok

$$\frac{\forall \mathsf{Dec} \in \mathsf{Decls}.\ \mathsf{Dec}\ \mathrm{Ok} \qquad \emptyset; \emptyset \vdash e : _}{\vdash \mathsf{Decls}\ e\ \mathrm{Ok}}$$

```
def ProgOk (d : Decls) (java : Bool) (e : Expr) : Prop
   :=
  (forall dec, List.Mem dec d -> ClassOk d java dec) /\
  exists t, Typing d java [] [] e t
```

Mapping. The _ type is the existential `t`.

A.5 Statements and High Level Proofs

```
def DeclsOk (d : Decls) : Prop := wfDecls d = true /\ forall dec, List.Mem dec d -> ClassOk d false dec
theorem stepWellFormed {d e e' t} (hd : DeclsOk d) (he : wfExpr e = true)
    (ht : Typing d false [] [] e t) (hs : Step d e e') : wfExpr e' = true
theorem progress {d e t} (hd : DeclsOk d) (he : wfExpr e = true)
    (ht : Typing d false [] [] e t) :
    e.isValue = true \/ exists e', Step d e e'
theorem preservation {d e e' t} (hd : DeclsOk d) (he : wfExpr e = true)
    (ht : Typing d false [] [] e t) (hs : Step d e e') : Typing d false [] [] e' t
theorem typeSoundness {d e0 e t} (hd : DeclsOk d) (he0 : wfExpr e0 = true)
    (ht : Typing d false [] [] e0 t) (hs : Steps d e0 e) :
    Typing d false [] [] e t /\ (e.isValue = true \/ exists e', Step d e e')
```

`progress`, `preservation` and `typeSoundness` are Progress, Preservation and Type Soundness of the section on the (un)soundness of method local classes. `DeclsOk d` is the global Decls such that Dec Ok for every Dec ∈ Decls, together with A1-A11 on Decls. `stepWellFormed` has no counterpart in the paper (Section A.3). `typeSoundness` is proved by induction on `Steps`, from the other three theorems. `progress` and `preservation` are proved from the low-level theorems with the same names, through these adequacy lemmas:

```
theorem isValue_tr {e : High.Expr} : (trExpr e).isValue = e.isValue
theorem wfExpr_tr {d : High.Decls} {e : High.Expr} (hd : High.wfDecls d = true)
    (he : High.wfExpr e = true) : Low.wfExpr (trExpr e) = true
theorem declsOk_tr {d : High.Decls} (hd : High.wfDecls d = true)
    (hok : forall dec, List.Mem dec d -> High.ClassOk d false dec) :
    Low.DeclsOk (trDecls d)
theorem typing_tr {d : High.Decls} {e : High.Expr} {t : High.Ty} (hd : High.wfDecls d = true) :
    High.Typing d false [] [] e t <-> Low.Typing (trDecls d) false [] [] (trExpr e) (trTy t)
theorem step_tr {d : High.Decls} {e e' : High.Expr} (hd : High.wfDecls d = true)
    (he : High.wfExpr e = true) (hs : High.Step d e e') :
    Low.Step (trDecls d) (trExpr e) (trExpr e')
theorem step_back {d : High.Decls} {e : High.Expr} {e'' : Low.Expr} (hd : High.wfDecls d = true)
    (he : High.wfExpr e = true) (hs : Low.Step (trDecls d) (trExpr e) e'') :
    exists e', High.Step d e e' /\ trExpr e' = e''
```

The adequacy lemmas, the translation and the low level can change together. What must not change is the high level and the statements of `lean/Funnelling/Soundness.lean`. While the two levels are identical, each adequacy lemma is an induction showing that the translation commutes with every function and preserves every judgement. The planned proofs of the low-level `progress` and `preservation`, and of `stepWellFormed`, are:

1. **Basic lemmas.** `Ty.beq` is equality; `lookup` on a substitution built by `mkSubst`; by A1, `declaredKs`, `scope` and `generic` have one entry per class name.
2. **Renaming.** `Expr.freshen` renames bound names consistently, so typing, kinding and well-formedness are invariant under it; the result binds only names of `ns`.
3. **Weakening** of Γ and Δ, for typing, kinding and subtyping.
4. **Substitution of type variables.** If Δ, Δ' types e, and ground types Gs satisfy the bounds Δ', then $e[\Delta' = Gs]$ is typed under Δ with the substituted type. Kinding and subtyping are preserved, but the checked cast is not: in rule CAST-VAR, after X becomes G, the source G may not reach the target through CAST-UP or CAST-DOWN. For the typing of rule IF-INSTANCEOF the lemma uses rule SUBS to lift e_0 from G to the substituted bound of X, and applies CAST-DOWN from there.
5. **Substitution of variables.** Values are closed and typed under $\emptyset;\emptyset$, so substituting values for variables of the same types preserves typing; `this` is not substituted into the methods of a local class.
6. **Inversion** of typing for each form of expression, through rule SUBS.
7. **Canonical forms.** A value of type C<Ts> has a class that is a subclass of C.
8. **Copies of a local class.** By consistent, the methods of a copy have the signatures of the declaration with the funnelled type variables substituted; so rules CALL and SUB-INHERIT-LOCAL, which read the declaration, agree with rule INNERFOUND, which runs the copy.
9. **Checked cast.** If $\emptyset \vdash \mathsf{typeOf}(v) <: C$<_>, v has type T_0 and $\emptyset \vdash T_0 \rightsquigarrow C$<$Ts$>, then v has type C<Ts>. The proof uses that the chain of superclasses is unique and that $Xs \subseteq \mathsf{FTV}(Ts')$ fixes Ts.
10. **stepWellFormed** by induction on `Step`: the fresh names keep A7, and the other requirements are local.
11. **progress** by induction on typing, with canonical forms; a call terminates its search at a class declaring the method, before `Object`, since typing found the method in a superclass.
12. **preservation** by induction on `Step`, with the lemmas above.

A.6 Validation Examples

`lean/Funnelling/Examples.lean` defines the class tables below and the examples, on the high level. A Lean theorem marked *proved* is accepted by Lean without `sorry`; a theorem marked *stated* has `sorry`, and needs the inversion lemmas of Section A.5.

```
class Object<> extends Object<>{}
class A<> extends Object<>{}
class B<> extends Object<>{ B<> onlyB(){ return new B<>(); } }
class Box<X extends Object<>> extends Object<>{ X val; X get(){ return this.val; }
    }
class C1<> extends Object<>{ A<> m(){ return new A<>(); } }
class C2<> extends C1<>{} // d1: the classes above
class Foo<X extends Object<>> extends Object<>{ X x(){ return this.x(); } }
class Maker<> extends Object<>{ <Y extends Object<>> Foo<Y> mk(Y y){ return
  new MyFoo<Y funnel> extends Foo<Y>(){ Y x(){ return y; } }; } } // d2: Object, A,
       B, Foo, Maker
class Broken<> extends Object<>{ <X extends Object<>> Foo<X> m(Object<> other, X x)
    { return
  if (other instanceof MyFoo<> w){ w }
  else { new MyFoo<> extends Foo<X>(){ X x(){ return x; } } }; } } // d3: Object, A
      , B, Foo, Broken
```

```
class Leaker<> extends Object<>{
  <Y extends Object<>> MyFoo2<> mk2(Y y){ return new MyFoo2<> extends Foo<Y>(){ Y x
      (){ return y; } }; }
  <Y extends Object<>> Foo<Y> leak(MyFoo2<> z){ return z; } } // d4: Object, A, B,
      Foo, Leaker
```

W1 `w1`, proved: d1 satisfies A1-A11.
W2 `w2`, proved: a method with two parameters `x` violates A4.
W3 `w3`, proved: an `if` binding the parameter `x` again violates A7.
W4 `w4`, proved: an undeclared superclass violates A9 and A10.
W5 `w5`, proved: d2 satisfies A1-A11.
W6 `w6`, proved: d3 and the main expression of Example 1 satisfy A1-A11.
W7 `w7`, proved: d4 satisfies A1-A11.
R1 `r1`, proved: `new Box<A<>>(new A<>()).val` → `new A<>()` by simpleFieldFound.
R2 `r2`, proved: `new C2<>().m()` → `new C2<>().[C2<>]m()` by simpleAddSuper.
R3 `r3`, proved: `new C2<>().[C2<>]m()` → `new C2<>().[C1<>]m()` by searchSuper.
R4 `r4`, proved: `new C2<>().[C1<>]m()` → `new A<>()` by simpleFound.
R5 `r5`, proved: R1 inside the receiver of a call, by ctx.
R6 `r6Then`, proved: `if (new A<>()instanceof A<> x){ x } else { new B<>()}` → `new A<>()` by ifThen.
R6 `r6Else`, stated: the same with `B<>` as the test → `new B<>()` by ifElse.
R7 `r7`, proved: `new Maker<>().[Maker<>]<A<>>mk(new A<>())` → `new MyFoo<A<>> extends Foo<A<>>(){ A<> x(){ return new A<>(); } }` by simpleFound, renaming `Y` and `y` into `Y1` and `y1`.
R8 `r8`, proved: the value of R7 `.x()` → `new A<>()` by innerFound.
R9 `r9`, proved: the value of R7 `.other()` → `.[Foo<A<>>]other()` by innerSuper.
R10 `r10`, proved: the value of R7 `.f` → `new Foo<A<>>().f` by innerFieldSuper.
R11 `r11`, proved: R2, R3, R4 in sequence.
T1 `t1`, proved: `new Box<A<>>(new A<>()).val : A<>`.
T2 `t2`, stated: `new Box<A<>>(new B<>())` is ill typed: `B<>` is not a subtype of `A<>`.
C1 `c1`, proved: `X extends Object<>` $\vdash$ `Foo<X>` $\rightsquigarrow$ `MyFoo<X>` in d2.
C2 `c2`, stated: not `X extends Object<>` $\vdash$ `Object<>` $\rightsquigarrow$ `MyFoo<X>` in d2.
M1 `m1`, stated: class `Maker` is Ok under funnelled typing.
J1 `j1`, stated: Example 1 (d3) is well typed under Java typing.
J2 `j2`, stated: Example 1 reduces to `new A<>().onlyB()`, which is stuck.
N1 `n1`, stated: Example 1 is ill typed under funnelled typing.
L1 `l1`, stated: class `Leaker` is Ok under Java typing.
L2 `l2`, stated: class `Leaker` is not Ok under funnelled typing.

In the paper, the verdict of each example follows from the rule named in the list, or, for W1-W7, from the requirements named in the list. For R7, the paper renames with fresh the type variable `Y` and the parameter `y` of `mk`, and then substitutes `A<>` and `new A<>()`; the method `x` of the local class binds no names. For T2, rule new-top needs `new B<>(): A<>`, and no rule gives it. For C2, rule cast-down needs `X` $\in$ FTV($\emptyset$), since `MyFoo<Y>` has the supertype `Object<>` with no type arguments. For J1, J2 and N1, Section 4.10 gives the derivation; N1 fails at $\emptyset \vdash$ `Foo<X>` Ok in rule new-local, since `MyFoo` funnels nothing. For L1 and L2, Section 4.10 discusses the method `leak`; L2 fails at the local class in `mk2`, as N1 does.

References

1 Nada Amin and Ross Tate. Java and scala's type systems are unsound: The existential crisis of null pointers. In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, OOPSLA 2016, page 838–848, New York, NY, USA, 2016. Association for Computing Machinery. doi:10.1145/2983990.2984004.

2 Manuel MT Chakravarty, Gabriele Keller, Simon Peyton Jones, and Simon Marlow. Associated types with class. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 1–13, 2005.

3 Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler. Featherweight java: A minimal core calculus for java and gj. *ACM Trans. Program. Lang. Syst.*, 23(3):396–450, 5 2001. doi:10.1145/503502.503505.

4 Marco Servetto, Nick Webster, and Colin S. Gordon. Java method-local inner classes are unsound. Talk at UNSOUND 2024, co-located with SPLASH 2024, October 2024. Recording at https://www.youtube.com/watch?v=DfkOpk733dQ. URL: https://2024.splashcon.org/details/unsound-2024-papers/3/Java-Method-Local-Inner-Classes-are-Unsound.